Predicting Flow Reversals in a Computational Fluid Dynamics Simulated Thermosyphon using Data Assimilation

A Thesis Presented

by

Andrew Reagan

to

The Faculty of the Graduate College

of

The University of Vermont

In Partial Fullfillment of the Requirements
for the Degree of Master of Science
Specializing in Applied Mathematics

January, 2014

Accepted by the Faculty of the Graduate College, The University of Vermont, in partial fulfillment of the requirements for the degree of Master of Science, specializing in Applied Mathematics.

Thesis Examination Committee:

______________________________ Advisor
Chris Danforth, Ph.D.

____
Peter Dodds, Ph.D.

______________________________ Chairperson
Darren Hitt, Ph.D.

______________________________ Dean, Graduate College
Cynthia J. Forehand, Ph.D.

Date: November 15, 2013

Abstract

A thermal convection loop is a circular chamber filled with water, heated on the bottom half and cooled on the top half. With sufficiently large forcing of heat, the direction of fluid flow in the loop oscillates chaotically, forming an analog to the Earth's weather. As is the case for state-of-the-art weather models, we only observe the statistics over a small region of state space, making prediction very difficult. To overcome this challenge, data assimilation methods, and specifically ensemble methods, use the computational model itself to estimate the uncertainty of the model to optimally combine these observations into an initial condition for predicting the future state. First, we build and verify four distinct DA methods. Then, a computational fluid dynamics simulation of the loop and a reduced order model are both used by these DA methods to predict flow reversals. The results contribute to a testbed for algorithm development.

in dedication to

my parents, Donna and Kevin Reagan, for their unwavering support

Acknowledgements

I would like to thank professors Chris Danforth and Peter Dodds for their outstanding advising. I would like to thank my fellow students for all of their help and toleration through the past two years. And finally I would like to thank my girlfriend Sam Spisiak, whose love and support make this thesis worth writing.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

In this chapter we explore the current state of numerical weather prediction, in particular data assimilation, along with an introduction to computational fluid dynamics and reduced order experiments.

1.1 Introduction

Prediction of the future state of many complex systems is integral to the functioning of our society. Some of these systems include weather (Hsiang et al. 2013), health (Ginsberg et al. 2008), the economy (Sornette and Zhou 2006), marketing (Asur and Huberman 2010) and engineering (Savely et al. 1972). For weather in particular, this prediction is made using supercomputers across the world in the form of numerical weather model integrations taking our current best guess of the weather into the future. The accuracy of these predictions depend on the accuracy of the models themselves, and the quality of our knowledge of the current state of the atmosphere.

Model accuracy has improved with better meteorological understanding of weather processes and advances in computing technology. To solve the initial value problem, techniques developed over the past 50 years are now broadly known as *data assimilation*. Formally, data assimilation is the process of using all available information, including short-range model forecasts and physical observations, to estimate the current state of a system as accurately as possible (Yang et al. 2006).

We employ a toy climate experiment as a testbed for improving numerical weather prediction algorithms, focusing specifically on data assimilation methods. This approach is akin to the historical development of current methodologies, and provides a tractable system for rigorous analysis. The experiment is a thermal convection loop, which by design simplifies our problem into the prediction of mostly convection. The dynamics of thermal convection loops have been explored under both periodic (Keller 1966) and chaotic (Welander 1995, Creveling et al. 1975, Gorman and Widmann 1984, Gorman et al. 1986, Ehrhard and Müller 1990, Yuen and Bau 1999, Jiang and Shoji 2003, Burroughs et al. 2005, Desrayaud et al. 2006, Yang et al. 2006, Ridouane et al. 2010) regimes. A full characterization of the computational behaivor of a loop under flux boundary conditions by Louisos et. al. deliminated four regimes: chaotic convection with reversals, high Ra aperiodic stable convection, steady stable convection, and conduction/quasi-conduction (Louisos et al. 2013). For the remainder of this work, we focus on the chaotic flow regime.

Computational simulations of the thermal convection loop are performed with the open-source finite volume C++ library OpenFOAM (Jasak et al. 2007). The open-source nature of this software enable it's integration with the data assimilation algorithms that were verified with a reduced order model.

1.2 History of NWP

The importance of Bjerknes in early developments in NWP is clearly described by Thompson's "Charney and the revival of NWP" (Thompson 1990):

> It was not until 1904 that Vilhelm Bjerknes - in a remarkable manifesto and testament of deterministic faith - started the central problem of NWP. This was the first explicit, coherent, recognition that the future state of the atmosphere is, *in principle*, completely determined by its detailed initial state and known boundary conditions, together with Newton's equations of motion, the Boyle-Charles-Dalton equation of state, the equation of mass continuity, and the thermodynamic energy equation. Bjerknes went further: he outlined an ambitious, but logical program of observation, graphical analysis of meterological data and graphical solution of the governing equations. He succeeded in persuading the Norwegians to support an expanded network of surface observation stations, founded the famous Bergen School of synoptic and dynamic meteorology, and ushered in the famous polar front theory of cyclone formation. Beyond providing a clear goal and a sound physical approach to dynamical weather prediction, V. Bjerknes instilled his ideas in the minds of his students and their students in Bergen and in Oslo, three of of whom were later to write important chapters in the development of NWP in the US (Rossby, Eliassen, and Fjörtoft).

It wasn't until 1922 that Lewis Richardson suggested a practical way to solve these equations. Richardson used a horizontal grid of about 200km, and 4 vertical layers in a computational mesh of Germany and was able to solve this system by hand (Richardson and Chapman 1922). Using the observations available, he computed the time derivative of pressure in the central box, predicting a change of 146 hPa, whereas in reality there was almost no change. This huge discrepancy was due mostly to the fact that his initial conditions (ICs) were not balanced, and therefore included fast-moving gravity waves which masked the true climate signal (Kalnay 2003). If the integration had continued, it would have "exploded," due to the Courant number induced by his mesh and timestep, which was greater than 1. We will revisit this condition in Chapter 3, but a simple explanation is as follows. The Courant number relates the movement of a quantity to the size of the mesh, written as

$$C = \frac{u_x \Delta t}{\Delta x}$$

in one dimension, and a value > 1 for a finite difference method solving partial different conditions is divergent.

Early on, the problem of fast travelling waves from unbalanced IC was solved by filtering the equations of motion based on the quasi-geostrophic (slow time) balance. Although this made running models feasible on the computers available during WW2, the incorporation of the full equations proved necessary for more accurate forecasts to be made.

The shortcomings of our understanding of the subgrid-scale phenomena do impose a limit on our ability to predict the weather, but this is not the upper limit. The upper limit of predictability exists for even a perfect model, as Edward Lorenz showed in with a three variable model found to exhibit sensitive dependence on initial conditions (Lorenz 1963).

Numerical weather prediction has since constantly pushed the boundary of computational power, and required a better understanding of atmospheric processes to make better predictions. In this direction, I make use of advanced computational resources in the context of a simplified atmospheric experiment to improve prediction skill.

1.3 IC determination: data assimilation

The purpose of data assimilation in weather prediction is defined by Talagrand as "using all the available information, to determine as accurately as possible the state of the atmospheric (or oceanic) flow." (Talagrand 1997) Broadly, data assimilation algorithms are used from areas as disparate as quadcopter stabilization (Achtelik et al. 2009) to the tracking of ballistic missle re-entry (Siouris et al. 1997). One such data assimilation algorithm, the Kalman filter, was originally implemented in the navigation system of Apollo program (Kalman and Bucy 1961, Savely et al. 1972).

Data assimilation algorithms consist of a 3-part cycle: predict, observe, and assimilate. Formally, the data assimilation problem is solved by minimizing the error in the presence of specific constraints. The prediction step involves making a prediction of the future state of the system, as well as the error of the model, in some capacity. Observing various systems comes in many flavors: rawindsomes and satellite irradiance for the atmosphere, temperature and velocity reconstruction of experiments from simple sensors, or sampling the market in finance. Assimilation is the combination of these observations and the predicitive model in such a way that minimizes the error of the future prediction based on this analysis.

The difficulties of this process arise in many fashions, and are addressed by parameter adjustments of existing algorithms or new approaches altogether. The first problem is ensuring the analysis respects the core balances of the model. In the atmosphere this is the balance of Coriolis force and pressure gradient (geostrophic balance), and in other systems this can arise in the conservation of physical (or otherwise) quantities.

In real applications, observations are made with irregular spacing in time and often can be made much more frequently than the length of data assimilation cycle. However the incorporation of more observations will not always improve forecasts: in the case where these observations can only be incorporated at the assimilation time the forecasts can degrade in quality (Kalnay et al. 2007). Algorithms which are capable of assimilating observations at the time they are made have a distinct advantage in this respect. In addition, the observations are not always direct measurements of the variables of the model. This can create non-linearities in the observation operator, the function that transforms observations in the physical space into model space. In Chapter 2 we will see where a non-linear operator complicates the minimization of the analysis error.

As George Box reminds us, "all models are wrong" (Box et al. 1978). The incorporation of model bias into the assimilation becomes important with imperfect models, and although this issue is well-studied, it remains a concern (Allgaier et al. 2012).

The final difficulty we need to address is the computational cost of the various algorithms. The complexity of models creates issues for both the computation speed and storage requirements, an example of each follows. The direct computation of the model covariance can be performed by integrating a linear tangent model which propagates errors in the model forward in time, but integrating this model amounts to n^2 model integrations where n is the number of variables in the model. For large systems, e.g. $n = 10^10$ for global forecast models, this is well beyond reach. Storing a covariance matrix at the full rank of model variables is not possible even for a model of the experiment we will address, which has 216,000 model variables. Workarounds for these specific problems include approximating the covariance without updating it or updating it using an ensemble of model runs, and reformulating the assimilation problem such that the covariance is only computed in observation space, which is typically orders of magnitude smaller in general.

1.4 NWP and fluid dynamics: computational models

Derived from Newton's law of motion, the basic equations governing flow of a viscous, heat-conducting fluid have been known since as early as 1823 (Navier 1823) and in their current form since 1846 (Stokes 1846). These are the Navier-Stokes equations. For the atmosphere, we can make various approximations to simplify the problem to arrive at what are known of equations of Geophysical Fluid Dynamics, which differ mainly in the incorporation of the Coriolis effect, and the neglecting of vertical component of advection. In both cases, the principle equation is the momentum equation, supplemented by the continuity and energy equations.

As stated, my interest here is not the accuracy of computational fluid dynamic simulations themselves, but rather their incorporation into the prediction problem. For this reason, the depth of exploration of available models is limited.

We chose to perform simulations in the open-source C++ library OpenFOAM. Because the code is open-source, I can modify it as necessary for data assimilation, and of particular importance becomes the discretized grid.

There are two main directions in which I do explore the computational model: meshing and solving. To solve the equations of motion numerically, we first must discretize them in space by creating the computational grid which we refer to as a mesh. The importance of properly designing and characterizing the mesh is profound: without the use of unreliable turbulence models, and without a grid small enough to resolve all of the scales of turbulence, we inevitably fail to capture the effects of sub-grid scale phenomena. In weather models, this is a known problem and is an open area of research. Turbulence models that we explore include $k - \epsilon$ models, LES, and laminar approximations.

Secondly, the choice of solving technique is a source of numerical and algorithmic uncertainty. The schemes which OpenFOAM has available are known to be dissapative and are limited in scope. That is, small perturbations such as those from a random error perturbation to an ensemble member in the EnKF are damped away. Without properly correcting this problem, the ensemble does not adequately capture the uncertainties in observations to obtain an approximation of the model error growth.

We investigate these meshing and solving techniques in greater detail in Chapter 3.

1.5 Toy climate experiments

Mathematical and computational models span from simple to kitchen-sink, depending on the phenomenon of interest. Simple, or "toy", models and experiments are important to gain theoretical understanding of the key processes at play. An example of such a conceptual model used to study climate is the energy balance model, pictured below.

☐ cite picture (from Chris Jones' class)

With a simple model, it is possible to capture the key properties of simple concepts (e.g. the greenhouse effect in the above energy balance model) and for this reason they have been and continue to be employed throughout scientific disciplines. The "kitchen-sink" models, like the Earth System Models (ESMs) used to study climate, incorporate all of the known physical, chemical, and biological processes for the fullest picture of the system. In the ESM case, the simulations represent our computational laboratory.

We study a model that is more complex than the EBM, but is still analytically tractable, known as the Lorenz 1963 system (Lorenz 63 for short). In 1962, Saltzmann attempted

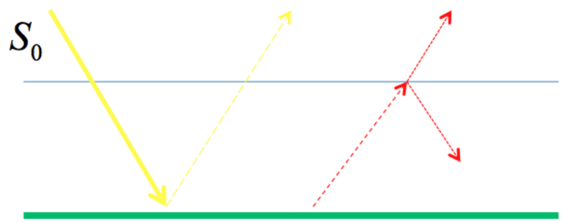

Figure 1.1: A schematic of the Energy Balance Model (EBM), a conceptual climate model based on the conservation of energy. S_0, represented by the yellow arrow, is energy coming from the sun that is absorbed by Earth's suface (horizontal green bar). The red, dashed arrow is energy radiating out of Earth's surface, of which some is reflected back by the atmosphere (horizontal grey line), and the remainder escapes. Since energy is conserved in the EBM, the parameterizations of surface albedo (amount reflected by Earth's surface), atmospheric absorbivity, and radiation rate determine the long term behavior of internal energy (temperature).

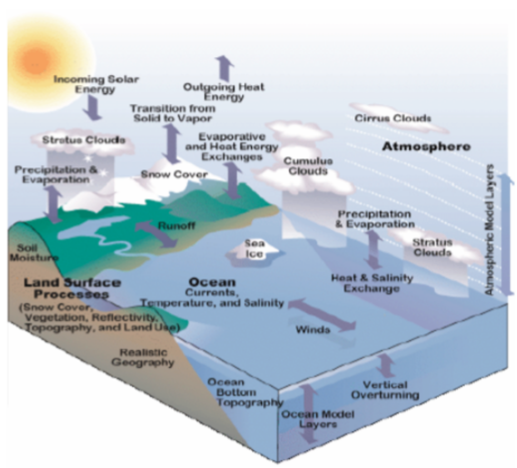

Figure 1.2: The "kitchen-sink" Earth System Model captures as many known processes as computationally feasible.

to model convection in a Rayleigh-Bérnard cell by reducing the equations of motion into their core processes (Saltzman 1962). Then in 1963 Edward Lorenz reduced this system ever further to 3 equations, leading to his landmark discovery of deterministic non-periodic flow (Lorenz 1963). These equations have since been the subject of intense study and have changed the way we view prediction and determinism, remaining the simple system of choice for examining nonlinear behaivor today (Kalnay et al. 2007).

Thermal convection loops are one of the few known experimental implementations of Lorenz's 1963 system of equations. Put simply, a thermal convection loop is a hula-hoop shaped tube filled with water, heated on the bottom half and cooled on the top half. As a non-mechanical heat pump, thermal convection loops have found applications for heat transfer in solar hot water heaters (Belessiotis and Mathioulakis 2002), cooling systems for computers (Beitelmal and Patel 2002), roads and railways that cross permafrost (Cherry et al. 2006), nuclear power plants (Detman 1968, Beine et al. 1992, Kwant and Boardman 1992), and other industrial applications. Our particular experiment consists of a plastic tube, heated on the bottom by a constant temperature water bath, and cooled on top by room temperature air.

Varying the forcing of temperature applied to the bottom half of the loop, there are three experimentally accessible flow regimes of interest to us: conduction, steady convection, and unsteady convection. The latter regime exhibits the much studied phenomenon of chaos in its changes of rotational direction. This regime can be described by the Lorenz 1963 system with the canonical parameter values listed in Table A.1. Edward Lorenz discovered and studied chaos most of his life, and it said to have jotted this simple description on a visit to the University of Maryland (Danforth 2009):

> Chaos: When the present determines the future, but the approximate present does not approximately determine the future.

In chapter 4, we present a derivation of Ehrhard-Müller Equations governing the flow in the loop following the work of (Harris et al. 2011). These equations are akin to the

Lorenz 63 system, as we shall see, and the experimental implementation is described in more detail in Chapter 4 as well.

In chapters 5 we present results for the predictive skill tests.

Finally, in chapter 6 we summarize the successes of this work and our contributions to the field. ☐ expand once these chapters are finished

Chapter 2

Data Assimilation

The goal of this chapter is to introduce and detail the practical implementation of state-of-the-art data assimilation algorithms. I derive the general form for the problem, and then talk about how each filter works. For a selection of these algorithms, I illustrate some of their properties via a MATLAB implementation using the Lorenz 1963 three-variable model:

$$\begin{aligned}
\frac{dx}{dt} &= \sigma(y - x) \\
\frac{dy}{dt} &= rx - y - xz \\
\frac{dz}{dt} &= xy - bz
\end{aligned}$$

The cannonical choice of $\sigma = 10, b = 8/3$ and $r = 28$ is used for all examples here, which produce the well known butterfly attractor (Figure 2.1):

From the simpler examples, I build the most advanced 4D-Var and 4DLETKF approaches with the full CFD model (216K variables).

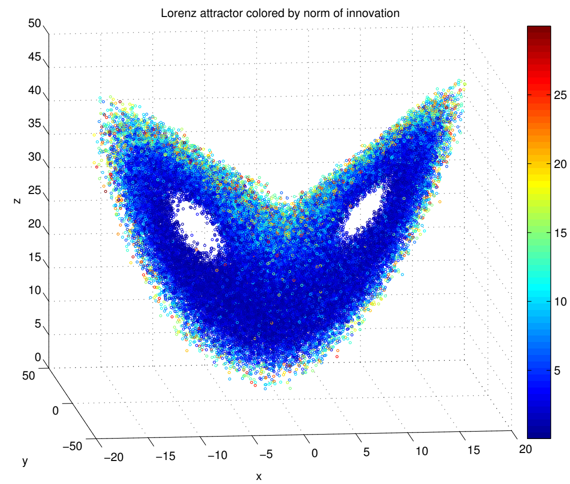

Figure 2.1: The Lorenz 63 attractor colored by the norm of the analysis increment. Larger numbers represent where forecasts are poor, and the data assimilation algorithm puts more weight on the observation when initializing a new forecast.

2.1 Basic derivation, OI

I begin by deriving the equations for optimal interpolation (OI) and for the Kalman Filter with one scalar observation. The goal of my derivation is to highlight which statistical information is useful, which is necessary, and what assumptions are generally made in the formulation of more complex algorithms.

Beginning with the one variable U, which could represent the horizontal velocity at one grid-point, say we make two independent observations of U: U_1 and U_2. If the true state of U is given by U_t, then the goal is to produce an analysis of $U_{1,2}$ which we call U_a, that is our best guess of U_t.

Denote the errors of $U_{1,2}$ by $\epsilon_{1,2}$ respectively, such that

$$U_{1,2} = U_t + \epsilon_{1,2}.$$

For starters, we assume that we do not know $\epsilon_{1,2}$. Next, assume that the errors vary around the truth, such that they are unbiased. This is written as

$$E(U_{1,2} - U_t) = E(\epsilon_{1,2}) = 0.$$

As of yet, our best guess of U_t would be the average $(U_1 + U_2)/2$, and for any hope of knowing U_t well we would need many unbiased observations at the same time. This is unrealistic, and to move forward we assume that we know the variance of each $U_{1,2}$. With this knowledge, which we write

$$E\left[\epsilon_{1,2}^2\right] = \sigma_{1,2}^2 \tag{2.1}$$

and further assume that these errors are uncorrelated

$$E\left[\epsilon_1 \epsilon_1\right] = 0 \tag{2.2}$$

which in the case of observing the atmosphere is at best hopeful.

We again attempt a linear fit of these observations to the truth, and can do better than an average. For U_a, our best guess of U_t, we write this linear fit as

$$U_a = \alpha_1 U_1 + \alpha_2 U_2. \tag{2.3}$$

If U_a is to be our best guess, we desire it to unbiased

$$E(U_a) = E(U_t) \Rightarrow \alpha_1 + \alpha_2 = 1$$

and to have minimal error variance σ_a^2.

At this point, note all of the assumptions that we have made: independent observations, unbiased instruments, knowledge of error variance, and uncorrelated observation errors. Of these, knowing the error variances (and in multiple dimensions, error covariance matrices) will become our focus, and better approximations of these errors have been largely responsible for recent improvements in forecast skill.

Solving 2.3 by substituting $\alpha_2 = 1 - \alpha_1$ have

$$\begin{aligned}
\sigma_a^2 &= E\left[\left(\alpha_1(U_1 - U_t) + (1-\alpha_1)(U_2 - U_t)\right)^2\right] \\
&= \alpha_1^2 E\left[\epsilon_1^2\right] + (1-\alpha_1^2)E\left[\epsilon_2^2\right] + 2\alpha_1(1-\alpha_1)E\left[\epsilon_1\epsilon_2\right] \\
&= \alpha_1^2\sigma_1^2 + (1-\alpha_1^2)\sigma_2^2
\end{aligned}$$

To minimize σ_a^2 with respect to α_1 we compute $\frac{\partial}{\partial\alpha_1}$ of the above and solve for 0 to obtain

$$\alpha_1 = \frac{\sigma_2^2}{2(\sigma_2^2 - \sigma_1^2)}$$

and similarly for α_2.

We can also state the relationship for σ_a^2 as a function of $\sigma_{1,2}^2$ by substituting our form for α_1 into the second step of 2.3 to obtain:

$$\frac{1}{\sigma_a^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}. \tag{2.4}$$

This last formula says that given accurate statistical knowledge of $U_{1,2}$, the precision of the analysis is the sum of the precision of the measurements.

We can now formulate the simplest data assimilation cycle by using our forecast model to generate a background forecast U_b and setting $U_1 = U_b$. We then assimilate the observa-

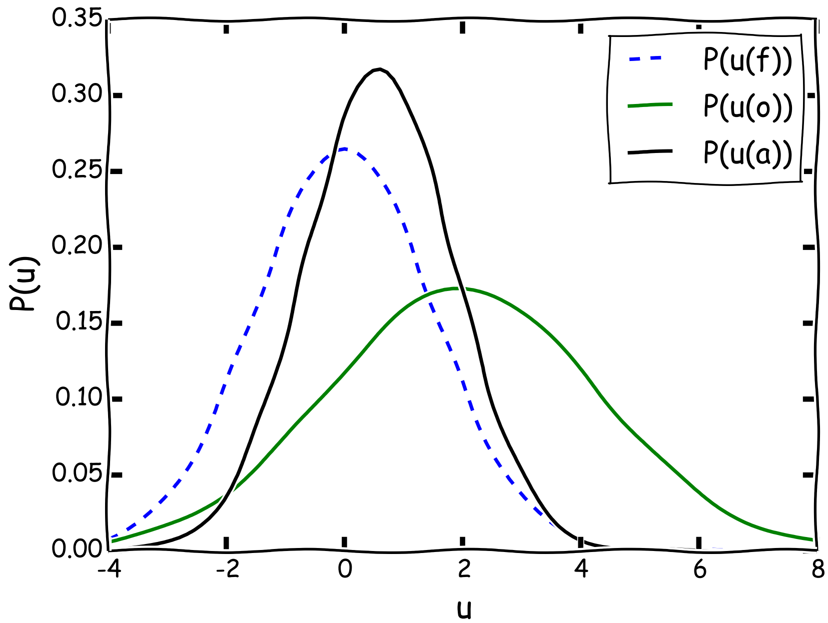

Figure 2.2: A 1-dimensional example of this data assimilation cycle. First, a forecast is made with mean 0 and variance 1, labeled u_f. An obvservation denoted u_o is the made with mean 2 and variance 2. From this information and Equation 2.6 we compute the analysis u_f with mean 2/3, noting the greater reliance on the lower variance input u_f.

tions U_o by setting $U_2 = U_o$ and solving for the analysis velocity U_a as

$$U_a = U_b + W(U_o - U_b). \tag{2.5}$$

The difference $U_o - U_b$ is commonly reffered to as the "innovation" and represents the new knowledge from the observations with respect to the background forecast. The optimal weight W is then given by 2.4 as

$$W = \frac{\sigma_b^2}{\sigma_b^2 - \sigma_o^2} = \sigma_b^2(\sigma_b^2 - \sigma_o^2)^{-1}. \tag{2.6}$$

The analysis error variance and precision are defined as before, and following Kalnay (2003) for convenience we can rewrite this in terms of W as:

$$\sigma_a^2 = (1 - W)\sigma_b^2. \tag{2.7}$$

We now generalize the above description for models of more than one variable to obtain the full equations for OI. This was originally done by Aerology et al. (1960) , and independently by Gandin (1965). The following notation will be used for the OI and the other multivariate filters. Consider a field of grid points, where there may be $\mathbf{x}_i = (p, p_{\text{rgh}}, T, u_x, u_y, u_z)$ variables at each point for pressure, hydrostatic pressure, temperature, and three dimensions of velocity. We combine all the variables into one list $\mathbf{x}$ of length n where n is the product of the number of grid points and the number of model variables. This list is ordered by grid point and then by variable (e.g. $\mathbf{x} = (p_1, T_1, p_2, T_2, \ldots, p_{n/2}, T_{n/2})$ for a field with just variables pressure p and temperature T). Denote the observations as $\mathbf{y}_0$, where the ordering is the same as $\mathbf{x}$ but the dimension is much smaller (e.g. by a factor of 10^3 for NWP) and the observations are not strictly at the locations of $\mathbf{x}$ or even direct measurements of the variables in $\mathbf{x}$. In our experiment, temperature measurements are in the model variable T, but are understood as the average of the temperature of the grid points surrounding it within some chosen radius. Examples of indirect and spatially heterogeneous observations in NWP are radar reflectivities and Doppler shifts, satellite radiances, and global positioning system atmospheric refractivities (Kalnay 2003).

We can now write the analysis with similar notation as for the scalar case with the addition of the observation operator H which transforms model variables into observation space:

$$\mathbf{x}_a = \mathbf{x}_b + \mathbf{W}\mathbf{d} \tag{2.8}$$

$$\mathbf{d} = \mathbf{y}_0 - H(\mathbf{x}_b) \tag{2.9}$$

$$\epsilon_a = \mathbf{x}_t - \mathbf{x}_a. \tag{2.10}$$

Note that H is not required to be linear, but here we write H as an $n_{\text{obs}} \times n$ matrix for a linear transformation.

Again, our goal is to obtain the weight matrix $\mathbf{W}$ that minimizes the analysis error ϵ_a. In Kalman Filtering, the weight matrix $\mathbf{W}$ is called the gain matrix $\mathbf{K}$. First, we assume that the background and observations are unbiased.

$$E\left[\epsilon_b\right] = E\left[\mathbf{x}_b\right) - E(\mathbf{x}_t\right] = 0$$

$$E\left[\epsilon_o\right] = E\left[\mathbf{y}_o\right] - E\left[H(\mathbf{x}_t)\right] = 0.$$

This a reasonable assumption, and if it does not hold true, we can account for this as a part of the analysis cycle (Dee and Da Silva 1998).

Define the forecast error covariances as

$$\mathbf{P}_a = \mathbf{A} = E\left[\epsilon_a \epsilon_a^T\right] \tag{2.11}$$

$$\mathbf{P}_b = \mathbf{B} = E\left[\epsilon_b \epsilon_b^T\right] \tag{2.12}$$

$$\mathbf{P}_o = \mathbf{R} = E\left[\epsilon_o \epsilon_o^T\right]. \tag{2.13}$$

Linearize the observation operator

$$H(\mathbf{x}+\partial\mathbf{x}) = H(\mathbf{x}) + \mathbf{H}\partial\mathbf{x} \tag{2.14}$$

where $h_{ij} = \partial H_i/\partial x_j$ are the entries of $\mathbf{H}$. Since the background is typically assumed to be a reasonably good forecast, it is convenient to write our equations in terms of changes to $\mathbf{x}_b$. The observational increment $\mathbf{d}$ is thus

$$\mathbf{d} = \mathbf{y}_o - H(\mathbf{x}_b) = \mathbf{y}_o - H(\mathbf{x}_t + (\mathbf{x}_b - \mathbf{x}_t)) \tag{2.15}$$

$$= \mathbf{y}_o - H(\mathbf{x}_t)\mathbf{H}(\mathbf{x}_b - \mathbf{x}_t) = \epsilon_o - \mathbf{H}\epsilon_b. \tag{2.16}$$

To solve for the weight matrix $\mathbf{W}$ assume that we know $\mathbf{B}$ and $\mathbf{R}$, and that their errors are uncorrelated:

$$E\left[\epsilon_o\epsilon_b^T\right] = E\left[\epsilon_b\epsilon_o^T\right] = 0. \tag{2.17}$$

We now use the method of best linear unbiased estimation (BLUE) to derive $\mathbf{W}$ in the equation $\mathbf{x}_a - \mathbf{x}_b = \mathbf{W}\mathbf{d}$, which approximates $\mathbf{x}_a - \mathbf{x}_b = \mathbf{W}\mathbf{d} - \epsilon_a$. Since $\mathbf{d} = \epsilon_o - \mathbf{H}\epsilon_b$ and the well known BLUE solution to this simple equation, the $\mathbf{W}$ that minimizes $\epsilon_a\epsilon_a^T$ is given by

$$\mathbf{W} = E\left[(\mathbf{x}_t - \mathbf{x}_b)(\mathbf{y}_o - \mathbf{H}\mathbf{x}_b)^T\right]\left(E\left[(\mathbf{y}_o - \mathbf{H}\mathbf{x}_b)(\mathbf{y} - \mathbf{H}\mathbf{x}_b)^T\right]\right)^{-1} \tag{2.18}$$

$$= E\left[(-\epsilon_b)(\epsilon_o - \mathbf{H}\epsilon_b)^T\right]\left(E\left[(\epsilon_o - \mathbf{H}\epsilon_b)(\epsilon_o - \mathbf{H}\epsilon_b)^T\right]\right)^{-1} \tag{2.19}$$

$$= E\left[-\epsilon_b\epsilon_o^T + \epsilon_b\epsilon_b^T\mathbf{H}^T\right]\left(E\left[\epsilon_o\epsilon_o^T - \mathbf{H}\epsilon_b\epsilon_o^T - \epsilon_o\epsilon_b^T\mathbf{H}^T + \mathbf{H}\epsilon_b\epsilon_b^T\mathbf{H}^T\right]\right)^{-1} \tag{2.20}$$

$$= \left(\cancel{E\left[-\epsilon_b\epsilon_o^T\right]} + E\left[\epsilon_b\epsilon_b^T\right]\mathbf{H}^T\right)\left(E\left[\epsilon_o\epsilon_o^T\right] - \cancel{\mathbf{H}E\left[\epsilon_b\epsilon_o^T\right]} - \cancel{E\left[\epsilon_o\epsilon_b^T\right]\mathbf{H}^T} + \mathbf{H}E\left[\epsilon_b\epsilon_b^T\right]\mathbf{H}^T\right)^{-1} \tag{2.21}$$

$$= \mathbf{B}\mathbf{H}^T\left(\mathbf{R} + \mathbf{H}\mathbf{B}\mathbf{H}^T\right)^{-1} \tag{2.22}$$

Now we solve for the analysis error covariance $\mathbf{P}_a$ using $\mathbf{W}$ from above and $\epsilon_a = \epsilon_b + \mathbf{W}\mathbf{d}$. This is (with the final bit of sneakiness using $\mathbf{W}\mathbf{R} - \mathbf{B}\mathbf{H}^T = -\mathbf{W}\mathbf{H}\mathbf{B}\mathbf{H}^T$):

$$\mathbf{P}_a = E\left[\epsilon_a\epsilon_a^T\right] = E\left[(\epsilon_b + \mathbf{Wd})(\epsilon_b + \mathbf{Wd})^T\right] \tag{2.23}$$

$$= E\left[(\epsilon_b + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b))(\epsilon_b + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b))^T\right] \tag{2.24}$$

$$= E\left[(\epsilon_b + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b))(\epsilon_b^T + (\epsilon_O^T - \epsilon_b^T\mathbf{H}^T)\mathbf{W}^T)\right] \tag{2.25}$$

$$= E\left[\epsilon_b\epsilon_b^T + \epsilon_b(\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)\mathbf{W}^T + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b)\epsilon_b^T + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b)(\epsilon_O^T - \epsilon_b^T\mathbf{H}^T)\mathbf{W}^T\right] \tag{2.26}$$

$$= E\left[\epsilon_b\epsilon_b^T\right] + E\left[\epsilon_b(\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)\right]\mathbf{W}^T + \mathbf{W}E\left[(\epsilon_o - \mathbf{H}\epsilon_b)\epsilon_b^T\right] + \mathbf{W}E\left[(\epsilon_o - \mathbf{H}\epsilon_b)(\epsilon_O^T - \epsilon_b^T\mathbf{H}^T)\right]\mathbf{W}^T \tag{2.27}$$

$$= \mathbf{B} - \mathbf{BH}^T\mathbf{W}^T - \mathbf{WHB} + \mathbf{W}E\left[\epsilon_o\epsilon_o^T - \cancel{\epsilon_o\epsilon_b^T}\mathbf{H}^T - \cancel{\mathbf{H}\epsilon_b\epsilon_o^T} + \mathbf{H}\epsilon_b\epsilon_b^T\mathbf{H}^T\right]\mathbf{W}^T \tag{2.28}$$

$$= \mathbf{B} - \mathbf{BH}^T\mathbf{W}^T - \mathbf{WHB} + \mathbf{WRW} + \mathbf{WHBH}^T\mathbf{W}^T \tag{2.29}$$

$$= \mathbf{B} - \mathbf{WHB} + (\mathbf{WR} - \mathbf{BH}^T)\mathbf{W}^T + \mathbf{WHBH}^T\mathbf{W}^T \tag{2.30}$$

$$= (\mathbf{I} - \mathbf{WH})\mathbf{B} \tag{2.31}$$

Note that in this formulation, we have assumed that $\mathbf{R}$ (the observation error covariance) accounts for both the instrumental error and the "representativeness" error, a term coined by Lorenc to describe the fact that the model does not account for subgrid-scale variability (Lorenc 1981). To summarize, the equations for OI are

$$\mathbf{W} = \mathbf{BH}^T\left(\mathbf{R} + \mathbf{HBH}^T\right)^{-1} \tag{2.32}$$

$$\mathbf{P}_a = (\mathbf{I} - \mathbf{WH})\mathbf{B} \tag{2.33}$$

As a final note, it is also possible to derive the "precision" of the analysis $\mathbf{P}_a^{-1}$ but in practice it is impractical to compute the inverse of the full matrix $\mathbf{P}_a$, so I omit this derivation. In general is also not possible to solve for $\mathbf{W}$ for the whole model, and localization of the solution is achieved by considering each grid point and the observations within some set radius of this. For different models, this equates to different assumptions on the correlations of observations, which I do not consider here.

2.2 3D-Var

Simply put, 3D-Var is the variational (cost-function) approach to finding the analysis. It has been shown that 3D-var solves the same statistical problem as OI (Lorenc 1986). The usefulness of the variational approach comes from the computational efficiency. This is why the 3D-Var analysis scheme became the most widely used data assimilation technique in operational weather forecasting, only to be supplemented by 4D-Var today.

The formulation of the cost function can be done in a Bayesian framework (Purser 1984) or using maximum likelihood (ML) approach (below). First, it is not difficult to see that solving for $\partial J/\partial U = 0$ for $U = U_a$ where we define $J(U)$ as

$$J(U) = \frac{1}{2}\left[\frac{(U-U_1)^2}{\sigma_1^2} + \frac{(U-U_2)^2}{\sigma_2^2}\right] \tag{2.34}$$

gives the same solution as obtained through least squares in our scalar assimilation. The formulation of this cost function obtained through the ML approach is simple (Edwards 1984). Consider the likelihood of true value U given an observation U_1 with standard deviation σ_1:

$$L_{\sigma_1}(U|U_1) = p_{\sigma_1}(U_1|U) = \frac{1}{\sqrt{2\pi}\sigma_1}e^{-\frac{(T_1-T)^2}{2\sigma_1^2}}. \tag{2.35}$$

The most likely value of U given two observations $U_{1,2}$ is the value which maximizes the joint likelihood (the product of the likelihoods), hence maximizes the logarithm of the joint likelihood (since the logarithm is increasing):

$$\max_U \ln L_{\sigma_1,\sigma_2}(U|U_1,U_2) = \max_U \left[a - \frac{(U-U_1)^2}{\sigma_1^2} - \frac{(U-U_2)^2}{\sigma_2^2} \right] \tag{2.36}$$

and this is equivalent to minimizing the cost function J above.

Jumping straight to result of Lorenc (Lorenc 1986) (we could define likelihood functions just like above), we define the multivariate 3D-Var as finding the $\mathbf{x}_a$ that minimizes the cost function

$$J(\mathbf{x}) = (\mathbf{x}-\mathbf{x}_b)^T\mathbf{B}^{-1}(\mathbf{x}-\mathbf{x}_b) + (\mathbf{y}_o + H(\mathbf{x}))^T\mathbf{R}(\mathbf{y}_o - H(\mathbf{x})). \tag{2.37}$$

The formal minimization of J can be obtained by solving for $\nabla_x J(\mathbf{x}_a)$, but since this is not useful in practice (iterative methods are used), we only present the result from (Kalnay 2003):

$$\mathbf{x}_a = \mathbf{x}_b + (\mathbf{B}^{-1} + \mathbf{H}^T\mathbf{R}^{-1}\mathbf{H})^{-1}\mathbf{H}^T\mathbf{R}^{-1}(\mathbf{y}_o - H(\mathbf{x}_b)). \tag{2.38}$$

The PSAS data assimilation is so similar to 3D-Var that I do not derive it here, the only difference being that the cost function J is solved in observation space. Since there are typically less observations than there are model variables, 3D-Var is a more efficient method.

Finally I note for 3D-Var that operationally $\mathbf{B}$ can be estimated with some flow dependence with the NMC or NCEP method, i.e.

$$\mathbf{B} \approx \alpha E\left[(\mathbf{x}_f(48h) - \mathbf{x}_f(24h))(\mathbf{x}_f(48h) - \mathbf{x}_f(24h))^T\right] \tag{2.39}$$

with on the order of 50 different short range forecasts.

2.3 Extended Kalman Filter

The Extended Kalman Filter (EKF) is the “gold standard” of data assimilation methods. It is the Kalman Filter “extended” for application to a nonlinear model. It works by updating our knowledge of error covariance matrix for each assimilation window, using a Tangent Linear Model (TLM), whereas in a traditional Kalman Filter the model itself can update the error covariance. The TLM is precisely the model (written as a matrix) that transforms a perturbation at time t to a perturbation at time $t+\Delta t$, analytically equivalent to the Jacobian of the model. Using the notation of Kalnay (Kalnay 2003), this amounts to making a forecast with the nonlinear model M, and updating the error covariance matrix $\mathbf{P}$ with the TLM L, and adjoint model L^T

$$\mathbf{x}^f(t_i) = M_{i-1}[\mathbf{x}^a(t_{i-1})]$$
$$\mathbf{P}^f(t_i) = L_{i-1}\mathbf{P}^a(t_{i-1})L_{i-1}^T + \mathbf{Q}(t_{i-1})$$

where $\mathbf{Q}$ is the noise covariance matrix (model error). In the experiments with Lorenz 63 presented in this section, $\mathbf{Q} = 0$ since our model is perfect. In NWP, $\mathbf{Q}$ must be approximated, e.g. using statistical moments on the analysis increments (Danforth et al. 2007, Li et al. 2009).

The analysis step is then written as (for H the observation operator):

$$\mathbf{x}^a(t_i) = \mathbf{x}^f(t_i) + \mathbf{K}_i\mathbf{d}_i \tag{2.40}$$

$$\mathbf{P}^a(t_i) = (\mathbf{I} - \mathbf{K}_i\mathbf{H}_i)\mathbf{P}^f(t_i) \tag{2.41}$$

where

$$\mathbf{d}_i = \mathbf{y}_i^o - \mathbf{H}[x^f(t_i)]$$

is the innovation.

The Kalman gain matrix is computed to minimize the analysis error covariance P_i^a as

$$\mathbf{K}_i = \mathbf{P}^f(t_i)\mathbf{H}_i^T[\mathbf{R}_i + \mathbf{H}_i\mathbf{P}^f(t_i)\mathbf{H}^T]^{-1}$$

where $\mathbf{R}_i$ is the observation error covariance.

Since we are making observations of the truth with random normal errors of standard deviation ϵ, then $\mathbf{R}$ is a matrix of just ϵ values.

The most difficult, and most computationally expensive, part of the EKF is deriving and integrating the TLM. For this reason, no operational weather centers use the EKF, although approximations such as the EnKF are promising (next).

2.3.1 Coding the TLM

The TLM is the model which advances an initial perturbation $\delta\mathbf{x}_i$ at timestep i to a final perturbation $\delta^*\mathbf{x}_{i+1}$ at timestep $i+1$. The dynamical system we are interested in, Lorenz

'63, is given as a system of ODE's:

$$\frac{d\mathbf{x}}{dt} = F(\mathbf{x}).$$

We integrate this system using a numerical scheme of our choice (here I used RK2), to obtain a model M discretized in time.

$$\mathbf{x}(t) = M[\mathbf{x}(t_0)].$$

Introducing a small perturbation $\mathbf{y}$, we can approximate the our model M of $\mathbf{x}(t_0) + \mathbf{y}(t_0)$ with a Taylor series around $\mathbf{x}(t_0)$, for which we assume to already have solved for:

$$\begin{aligned} M[\mathbf{x}(t_0) + \mathbf{y}(t_0)] &= M[\mathbf{x}(t_0)] + \frac{\partial M}{\partial \mathbf{x}}\mathbf{y}(t_0) + O[\mathbf{y}(t_0)^2] \\ &\approx \mathbf{x}(t) + \frac{\partial M}{\partial \mathbf{x}}\mathbf{y}(t_0). \end{aligned}$$

We can then solve for the linear evolution of the small perturbation $\mathbf{y}(t_0)$ as

$$\frac{d\mathbf{y}}{dt} = \mathbf{J}\mathbf{y}\mathbf{??} \tag{2.42}$$

where $\mathbf{J} = \partial F/\partial \mathbf{x}$ is the Jacobian of F. We can solve the above system of linear ordinary differential equations using the same numerical scheme as we did for the nonlinear model.

The problem with Equation **??** solving system of equations is that the Jacobian matrix of discretized code is not necessarily identical to the discretization of the Jacobian operator for the analytic system. This is a problem because we need to have the TLM of our model M, which is the time-space discretization of the solution to $d\mathbf{x}/dt = F(\mathbf{x})$. We can apply our

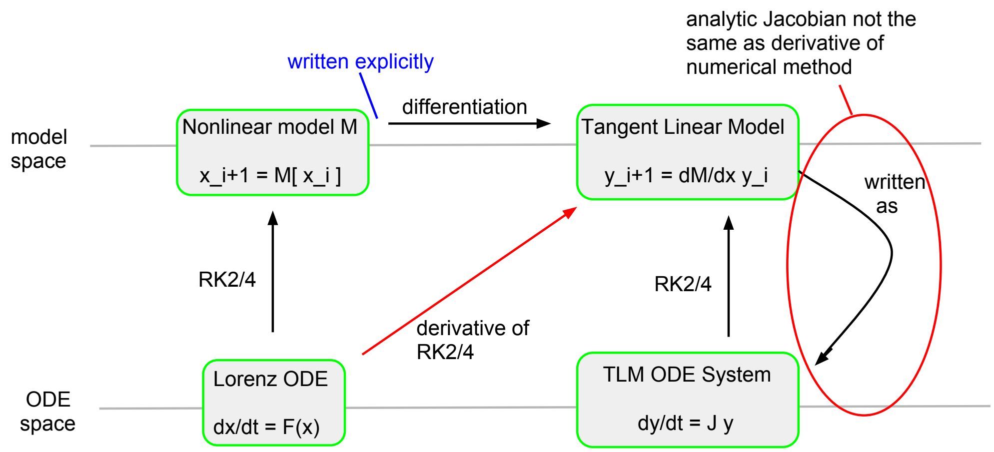

Figure 2.3: An explanation of how and why the best way to obtain a TLM is with a differentiated numerical scheme.

numerical method to the $d\mathbf{x}/dt = F(\mathbf{x})$ to obtain M explicitly, and then take the Jacobian of the result. But, that is extremely messy (and costly), since Runge-Kutta methods have nested function iterations. It is therefore desirable to take the derivative of the numerical scheme directly, and apply this differentiated numerical scheme to the system of equations $F(\mathbf{x})$ to obtain the TLM. A schematic of this scenario is illustrated in Figure 2.3. It is for this reason that there exists code that takes the derivative of numerical code (i.e. Fortran) for implementing the EKF on models larger than 3 dimensions with numerical schemes more complicated than RK2 or RK4.

In order to verify my implementation of the TLM, I propagate a small error in the Lorenz 63 system and plot the difference between that error and the TLM predicted error in each variable forward in time in Figure 2.4.

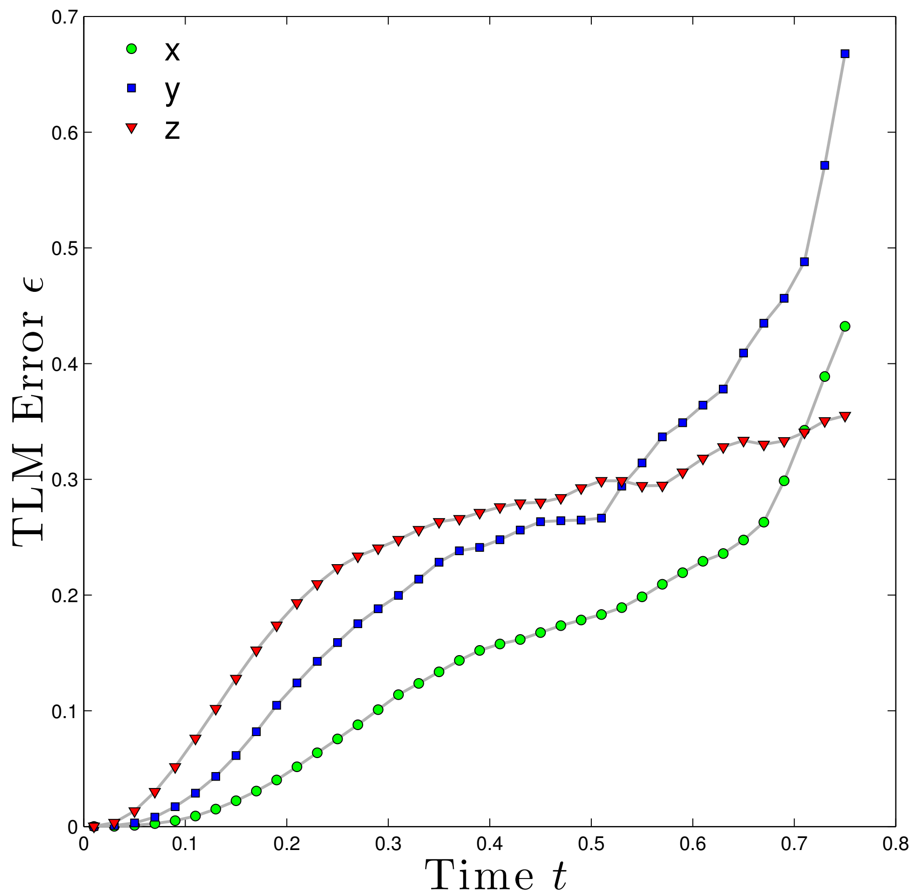

Figure 2.4: The future error predicted by the TLM is compared to the error growth in Lorenz 63 system for an initial perturbation with standard deviation of 0.1, averaged over 1000 TLM integrations. The ϵ is not the error predicted by the TLM, but rather the error of the TLM in predicting the error growth.

2.4 Ensemble Kalman Filter

The computational cost of the EKF is mitigated through the approximation of the error covariance matrix $\mathbf{P}_f$ from the model itself, without the use of a TLM.

One such approach is the use of a forecast ensemble, where a collection of models (ensemble members) are initialized from the prior analysis $\mathbf{x}_a$ with random error. The spread of these models with random errors is then used to estimate the error covariance. In the limit of infinite ensemble size, the EnKF is equivalent to the EKF (Evensen 2003). There are two prevailing methodologies for maintaining independent ensemble members: adding errors to the observation used by each forecast and assimilating each ensemble individually or distributing ensemble initial conditions from the analysis prediction. In the prior, the random error added to the observation can be truly random (Harris et al. 2011) or be drawn from a distribution related to the analysis covariance, which I chose for this type of filter. The latter method of distributing ensemble members around the analysis forecast, with initial conditions perturbed as (for example) the square root of the ensemble has advantages compared to perturbed observations (Evensen 2003). It has been show that the perturbed observation approach introduces additional sampling error that reduces the analysis covariance accuracy and increases the probability of of underestimating analysis error covariance (Hamill et al. 2001).

With a finite ensemble size this method is only an approximation and therefore in practice it often fails to capture the full spread of error. To better capture the model variance, additive and multiplicative inflation factors are used to obtain a good estimate of the error covariance matrix. The specific factors used are tuned through an exhaustive search, and included in the next section. The spread of ensemble members in the x_1 variable of the Lorenz model, as distance from the analysis, can be seen in Figure 2.5

The only difference between this approach and the EKF, in general, is that the forecast error covariance $\mathbf{P}^f$ is computed from the ensemble members, without the need for a tangent linear model:

$$\mathbf{P}^f \approx \frac{1}{K-2} \sum_{k \neq l} \left(\mathbf{x}_k^f - \overline{\mathbf{x}}_l^f\right) \left(\mathbf{x}_k^f - \overline{\mathbf{x}}_l^f\right)^T .$$

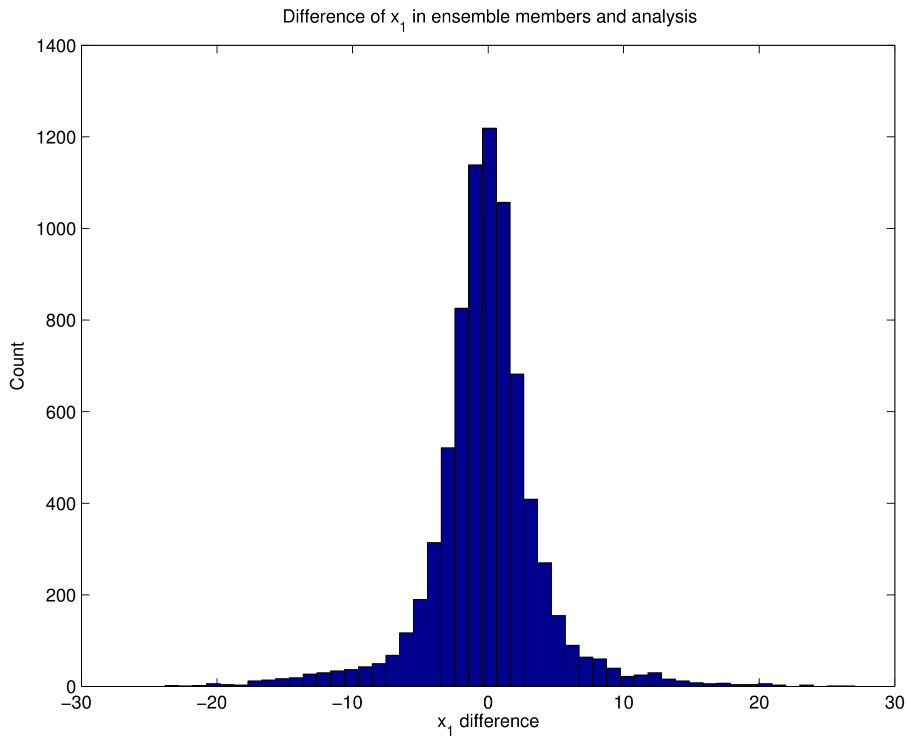

Figure 2.5: The difference of ensemble forecasts from the analysis is reported for 760 assimilation windows in one model run of length 200, with 10 ensemble members and an assimilation window of length 0.261. This has the same shape of as the difference between ensemble forecasts and the mean of the forecasts (not shown). This spread of ensemble forecasts is what allows us to estimate the error covariance of the forecast model, and appears to be normally distributed.

In computing the error covariance $\mathbf{P}_f$ from the ensemble, we wish to add up the error covariance of each forecast with respect to the mean forecast. But this would underestimate the error covariance, since the forecast we're comparing against was used in the ensemble average (to obtain the mean forecast). Computing the error covariance matrix for each forecast, that forecast itself is excluded from the ensemble average forecast.

We can see the classic spaghetti of the ensemble with this filter implemented on Lorenz 63 in Figure 2.6.

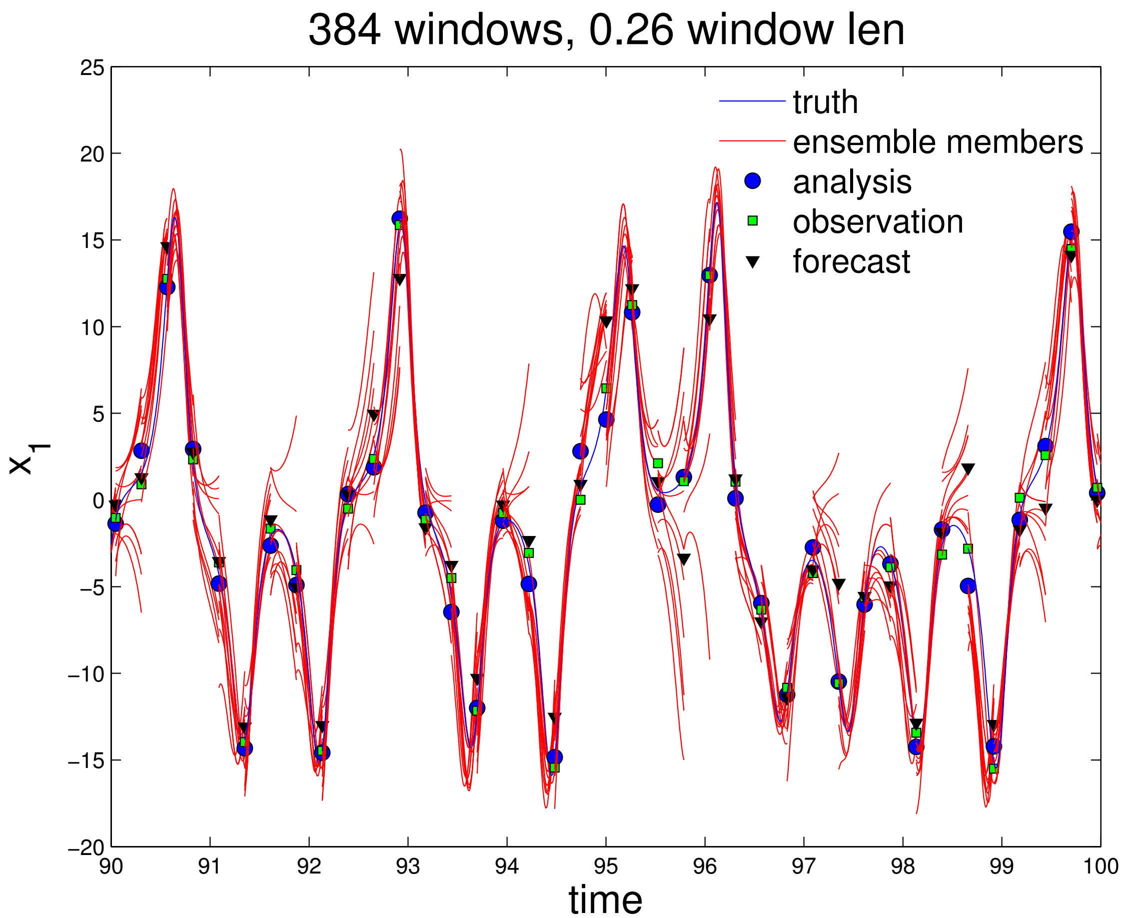

Figure 2.6: A sample time-series of the ensembles used in the EnKF. In all tests, as seen here, 10 ensemble members are used.

The forecast from an ensemble is the average of the ensemble forecasts, and an explanation for this choice is substantiated by Burgers (Burgers et al. 1998). The general EnKF which I use is most similar to that of Burgers. Many algorithms based on the EnKF have been proposed and include the Ensemble Transform Kalman Filter (ETKF) (Ott et al. 2004), Ensemble Analysis Filter (EAF) (Anderson 2001), Ensemble Square Root Filter (EnSRF) (**?**), Local Ensemble Kalman Filter (LEKF) (Ott et al. 2004), and finally the Local Ensemble Transform Kalman Filter (LETKF) (Hunt et al. 2007). I explore some of these in the following sections. A comprehensive overview through 2003 is provided by Evensen (Evensen 2003). Next, I present the results of a MATLAB implementation of the EKF and basic EnKF.

2.4.1 Performance of EnKF, EKF on Lorenz 63

With the EnKF and the EKF algorithms coded as above, I present some basic results on their performance. First, we see that using a data assimilation algorithm is highly advantageous in general (Figure 2.7). Of course we could think to use the observations directly when DA is not used, but since we only observing the x_1 variable (and in real situations measurements are not so direct), this is not prudent.

Next, we test the performance of each filter against increasing assimilation window length. As we would expect, longer windows make forecasting more challenging for both algorithms. The ensemble filter begins to out-perform the extended Kalman filter for longer window lengths because the ensemble more accurately captures the nonlinearity of the model.

Both filters are sensitive to the size of the covariance matrix, and in some cases to maintain the numerical stability of the assimilation it is necessary to use additive and/or multiplicative covariance inflation. In both the EKF and the EnKF mutliplicative error is performed with inflaction factor Δ by

$$\mathbf{K} \leftarrow (1 + \Delta)\mathbf{K}. \tag{2.43}$$

Additive inflation is performed for the EKF as in Yang et al (2006) and Harris et al (2011) where uniformly distributed random numbers in $[0, \mu]$ are added to the diagonal. This is

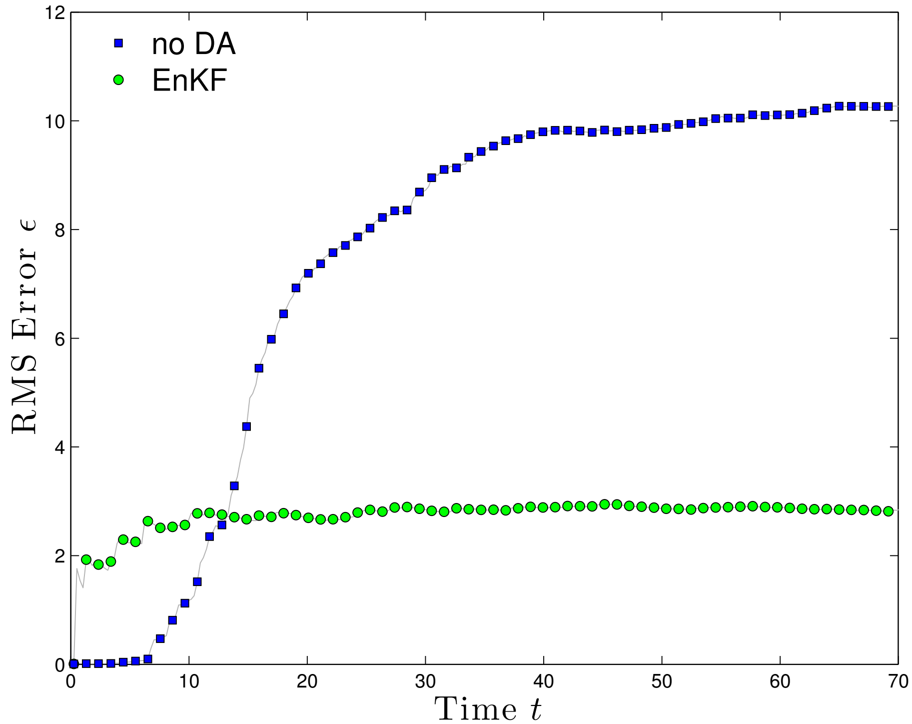

Figure 2.7: For near-perfect initial knowledge of the atmospheric state, the running average RMS error of a prediction using DA is compared against a prediction that does not. Initial error for DA is normal with mean 0 and variance 0.01, and the RMS errors reported are averages of 100 runs. Observational noise is normally distributed with mean 0 and variance 0.5 (approximately 5% of climatological variation), and an assimilation window length of 0.261 is used. As would be expected, in the long run DA greatly improves forecast skill, as forecasts without DA saturate to climatological error. The initial period where a forecast exceeds the skill of DA is potentially due to the spin-up time required by the filter to dial in the analysis error covariance.

accomplished in MATLAB by drawing a vector ν of length n of uniformly distributed numbers in $[0,1]$, multiplying it by μ, and adding it to the diagonal:

$$\mathbf{K} \leftarrow \mathbf{K} + \text{diag}(\mu \cdot \nu). \tag{2.44}$$

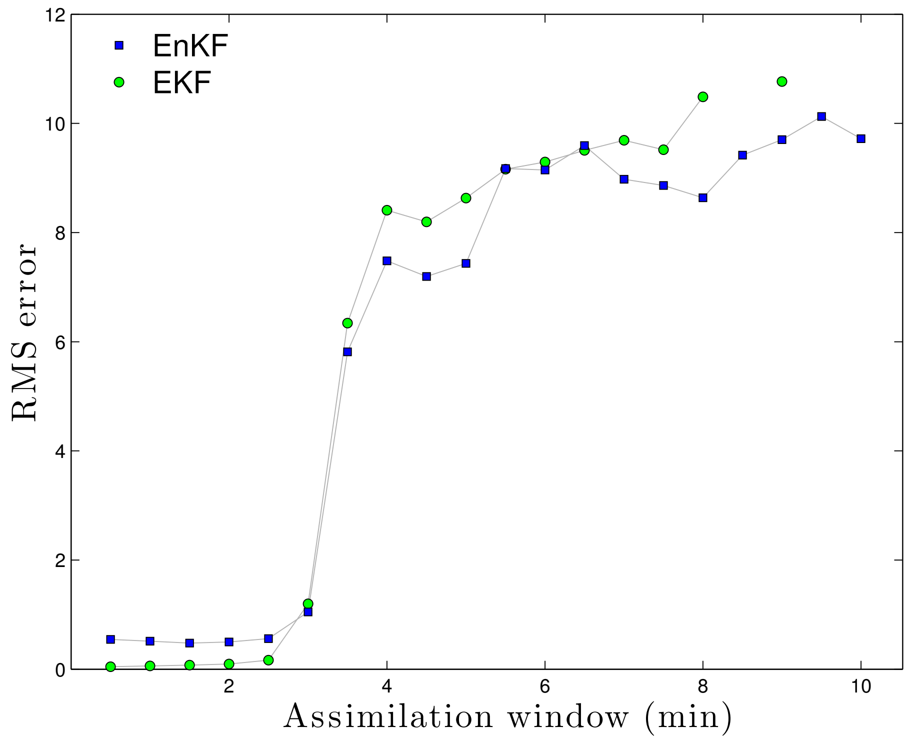

Figure 2.8: The RMS error (not scaled by climatology) is reported for our EKF and EnKF filters, measured as the difference between forecast and truth at the end of an assimiliation window for the latter 2500 assimiliation windows in a 3000 assimilation window model run. Error is measured in the only observed variable, x_1. Increasing the assimilation window led to an decrease in predictive skill, as expected. Additive and multiplicative covariance inflation is the same as Harris et. al (2011).

For the EnKF we perform additive covariance inflation to the analysis state itself for each ensemble:

$$\mathbf{x} \leftarrow \mathbf{x} + \nu. \tag{2.45}$$

The result of searching over these parameters for the optimal values to minimize the analysis RMS error differs for each window length, so here here we present the result for a window of length 210 seconds using the EKF. In Figure 2.9 we see that there is a clear region of parameter space for which the filter performs with lower RMS, although this region appears to extend into values of $\Delta > 3$.

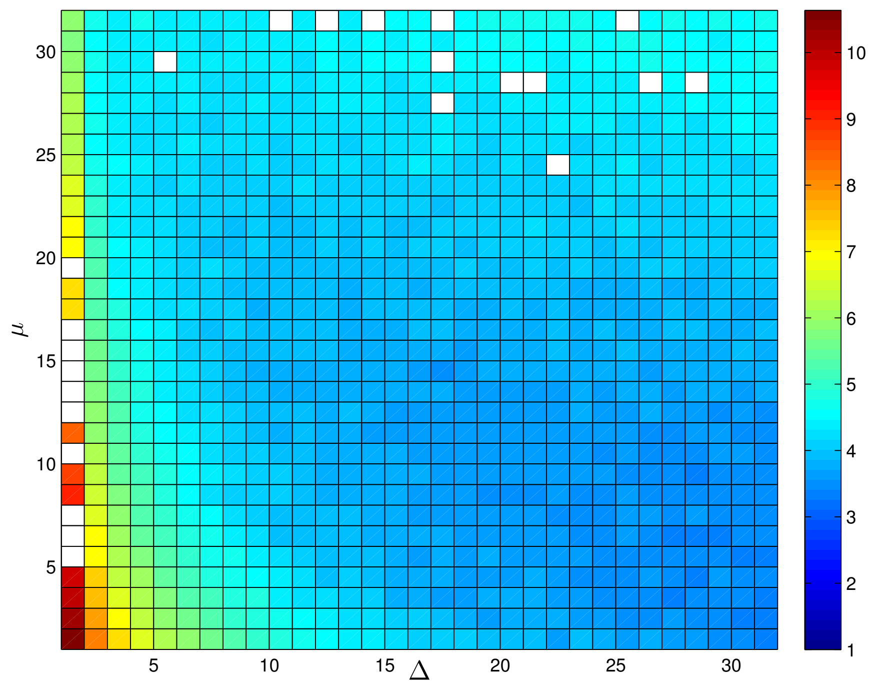

Figure 2.9: The RMS error averaged over 100 model runs of length 1000 windows is reported for the EKF for varying additive and multiplicative inflation factors Δ and μ. Each of the 100 model runs starts with a random IC, and the analysis forecast starts randomly. The filter performance RMS is computed as the RMS value of the difference between forecast and truth at the assimilation window for the latter 500 windows, allowing a spin-up of 500 windows.

2.4.2 Ensemble Transform Kalman Filter (ETKF)

The ETKF introduced by Bishop is one type of square root filter, and I present it here to provide background for the formulation of the LETKF (Bishop et al. 2001). For a square root filter in general, we begin by writing the covariance matrices as the product of their matrix square roots. Because $\mathbf{P}_a, \mathbf{P}_f$ are symmetric positive-definite (by definition) we can write

$$\mathbf{P}_a = \mathbf{Z}_a\mathbf{Z}_a^T \quad , \quad \mathbf{P}_f = \mathbf{Z}_f\mathbf{Z}_f^T \tag{2.46}$$

for $\mathbf{Z}_a, \mathbf{Z}_f$ the matrix square roots of $\mathbf{P}_a, \mathbf{P}_f$ respectively. We are not concerned that this decomposition is not unique, and note that $\mathbf{Z}$ must have the same rank as $\mathbf{P}$ which will prove computationally advantageous. The power of the SRF is now seen as we represent the columns of the matrix $\mathbf{Z}_f$ as the difference from the ensemble members from the ensemble mean, to avoid forming the full forecast covariance matrix $\mathbf{P}_f$. The ensemble members are updated by applying the model M to the states $\mathbf{Z}_f$ such that an update is performed by

$$\mathbf{Z}_f = M\mathbf{Z}_a. \tag{2.47}$$

Per (**?**), the solution for $\mathbf{Z}_a$ can be solved by perturbed observations

$$\mathbf{Z}_a = (\mathbf{I} - \mathbf{KH})\mathbf{Z}_f + \mathbf{KW} \tag{2.48}$$

where $\mathbf{W}$ is the observation perturbation of Gaussian random noise. This gives the analysis with the expected statistics

$$\langle \mathbf{Z}_a\mathbf{Z}_a^T \rangle = (\mathbf{I} - \mathbf{KH})\mathbf{P}_f(\mathbf{I} - \mathbf{KH})^T + \mathbf{KRK}^T \tag{2.49}$$

$$= \mathbf{P}_a \tag{2.50}$$

but as mentioned earlier, this has been shown to decrease the resulting covariance accuracy.

In the SRF approach, the “Potter Method” provides a deterministic update by rewriting

$$\mathbf{P}_a = \mathbf{Z}_a\mathbf{Z}_a^T = \left(\mathbf{I} - \mathbf{P}_f\mathbf{H}^T(\mathbf{R} + \mathbf{H}\mathbf{P}_f\mathbf{H}^T)^{-1}\mathbf{H}\right)\mathbf{P}_f \tag{2.51}$$

$$= \mathbf{Z}_f\left(\mathbf{I} - \mathbf{Z}_f\mathbf{H}^T(\mathbf{H}\mathbf{Z}_f\mathbf{Z}_f^T\mathbf{H}^T + \mathbf{R})^{-1}\mathbf{H}\mathbf{Z}_f\right)\mathbf{Z}_f^T \tag{2.52}$$

$$= \mathbf{Z}_f\left(\mathbf{I} - \mathbf{V}\mathbf{D}^{-1}\mathbf{Z}^T\right)\mathbf{Z}_f^T. \tag{2.53}$$

We have defined $\mathbf{V} \equiv (\mathbf{H}\mathbf{Z}_f)^T$ and $\mathbf{D} \equiv \mathbf{V}^T\mathbf{V} + \mathbf{R}$. Now for the ETFK step, we use the Sherman-Morrison-Woodbury identity to show

$$\mathbf{I} - \mathbf{Z}\mathbf{D}^{-1}\mathbf{Z}^T = (\mathbf{I} + \mathbf{Z}_f\mathbf{H}^T\mathbf{R}^{-1}\mathbf{H}\mathbf{Z}_f)^{-1}. \tag{2.54}$$

The matrix on the right hand side is practical to compute because it is of dimension of the rank of the ensemble, assuming that the matrix inverse $\mathbf{R}^{-1}$ is available. This is the case in our experiment, as the observation are uncorrelated. The analysis update is thus

$$\mathbf{Z}_a = \mathbf{Z}_f\mathbf{C}(\mathbf{\Gamma} + \mathbf{I})^{-1/2}, \tag{2.55}$$

where $\mathbf{C}\mathbf{\Gamma}\mathbf{C}^T$ is the eigenvalue decomposition of $\mathbf{Z}_f^T\mathbf{H}^T\mathbf{R}^{-1}\mathbf{H}\mathbf{Z}_f$. From (**?**) we note that the matrix $\mathbf{C}$ of orthonormal vectors is not unique.

Then the analysis perturbation is calculated from

$$\mathbf{Z}_a = \mathbf{Z}_f\mathbf{X}\mathbf{U}, \tag{2.56}$$

where $\mathbf{X}\mathbf{X}^T = (\mathbf{I} - \mathbf{Z}\mathbf{D}^{-1}\mathbf{V}^T)$ and $\mathbf{U}$ is an arbitrary orthogonal matrix.

In this filter, the analysis perturbations are assumed to be equal to the backgroud perturbations post-multiplied by a transformation matrix $\mathbf{T}$ so that that the analysis error covariance satisfies

$$\mathbf{P}_a = (1 - \mathbf{K}\mathbf{H})\mathbf{P}_f.$$

The analysis covariance is also written

$$\mathbf{P}_f = \frac{1}{P-1}(\mathbf{X}_a(\mathbf{X}_a)^T = \mathbf{X}^b\hat{\mathbf{A}}(\mathbf{X}^b)^T$$

where $\hat{\mathbf{A}} = [(P-1)\mathbf{I} + (\mathbf{H}\mathbf{X}^b)^T\mathbf{R}^{-1}(\mathbf{H}\mathbf{X}^b)]^{-1}$. The analysis perturbations are $\mathbf{X}^a = \mathbf{X}^b\mathbf{T}$ where $\mathbf{T} = [(P-1)\hat{\mathbf{A}}]^{1/2}$.

To summarize, the steps for the ETKF are to (1) form $\mathbf{Z}_f^T\mathbf{H}^T\mathbf{R}^{-1}\mathbf{H}\mathbf{Z}_f$, assuming $\mathbf{R}^{-1}$ is easy, and (2) compute its eigenvalue decomposition, and apply it to $\mathbf{Z}_f$.

2.4.3 Local Ensemble Kalman Filter (LEKF)

The LEKF implements a strategy that becomes very important for large simulations: localization. Namely, the analysis is computed for each grid-point using only local observations, without the need to build matrices that represent the entire analysis space. This localization removes long-distance correlations from $\mathbf{B}$ and allows greater flexibility in the global analysis by allowing different linear combinations of ensemble members at different spatial locations (Kalnay et al. 2007).

Difficulties are presented however, in the form of computational grid dependence for the localization procedure. The toroidal shape of our experiment, when discretized by OpenFOAM's mesher and grid numbering optimized for computational bandwidth for solving, does not require that adjacent grid points have adjacent indices. In fact, they are decidedly non-adjacent for certain meshes, with every other index corresponding to opposing sides of the loops. We overcome this difficulty by placing a more stringent requirement on the mesh of localization of indexing, and extracting the adjacency matrix for use in filtering.

The general formulation of the LEKF by Ott (Ott et al. 2004) goes as follows, quoting directly:

1. Globally advance each ensemble member to the next analysis timestep. Steps 2-5 are performed for each grid point

2. Create local vectors from each ensemble member

3. Project that point's local vectors from each ensemble member into a low dimensional subspace as represented by perturbations from the mean

4. Perform the data assimilation step to obtain a local analysis mean and covariance

5. Generate local analysis ensemble of states

6. Form a new global analysis ensemble from all of the local analyses

7. Wash, rinse, and repeat.

Figure 2.10 depicts this cycle in greater detail, with similar notations to that used thus far (Ott et al. 2004).

In the reconstructed global analysis from local analyses, since each of the assimilations were performed locally, we cannot be sure that there is any smoothness, or physical balance, between adjacent points. To ensure a unique solution of local analysis perturbations that respects this balance, the prior background forecasts are used to constrain the construction of local analysis perturbations to minimize the difference from the background.

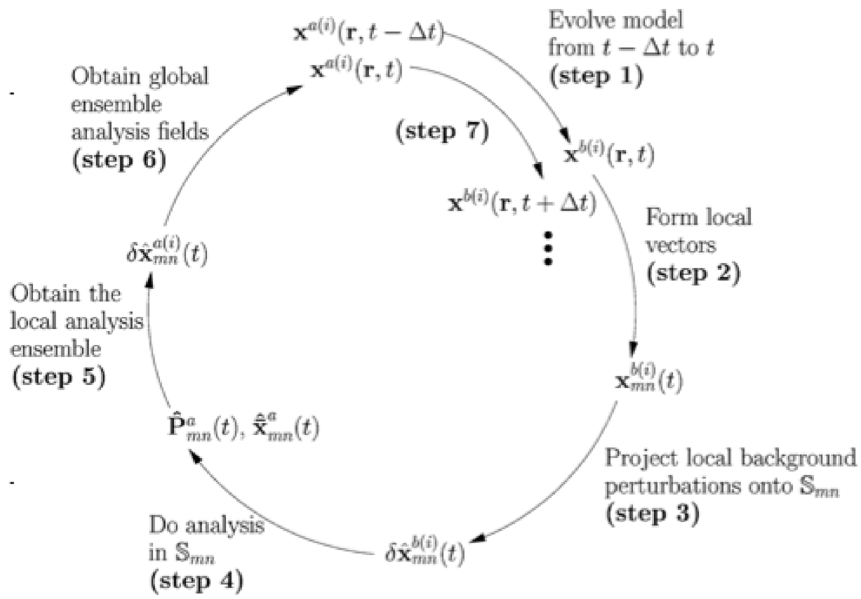

Figure 2.10: The algorithm for the LEKF as described by Ott (2004).

2.4.4 Local Ensemble Transform Kalman Filter (LETKF)

Proposed by Hunt in 2007 with the stated objective of computational efficiency, the LETKF is named from its most similar algorithms from which it draws (Hunt et al. 2007). With the formulation of the LEKF and the ETKF there is not much new to be said here, but rather a synthesis of the advantages of both of these approaches. This is the method that was sufficiently efficient for implementation on the full OpenFOAM CFD model of 216,000 model variables, and so I present it in more detail and follow the notation of Hunt et al (2007).

As in the LEKF, we explicitly perform the analysis for each grid point of the model. The choice of observations to use for each grid point can be selected a priori, and tuned adaptively. The simplest choice is to define a radius of influence for each point, and use observations within that. In addition to this simple scheme, Hunt proposes weighing observations by their proximity to smoothly transition across them. The goal is to avoid sharp boundaries between neighboring points.

Regarding the LETKF analysis problem as the construction of an analysis ensemble from the background ensemble, where we specify that the mean of the analysis ensemble minimizes a cost function, we state the equations in terms of a cost function. The LETKF is thus

$$\left\{\mathbf{x}_{b(i)} \,:\, i = 1..k\right\} \xrightarrow{\quad\text{LETKF}\quad} \left\{\mathbf{x}_{a(i)} \,:\, i = 1..k\right\} \tag{2.57}$$

where

$$\begin{aligned} J(\mathbf{x}) =& (\mathbf{x} - \overline{\mathbf{x}}_b)^T (\mathbf{P}_b)^T (\mathbf{x} - \overline{\mathbf{x}}_b) \\ & + (\mathbf{y}_o - H(\mathbf{x}))^T \, \mathbf{R}^{-1} \, (\mathbf{y}_o - H(\mathbf{x})) \,. \end{aligned}$$

Since we have defined $\mathbf{P}_b = (1/(k-1))\mathbf{X}_b\mathbf{X}_b^T$ for the $m \times k$ matrix $\mathbf{X}_b = (\mathbf{x}_{b(1)}, \ldots, \mathbf{x}_{b(k)})$, the rank of $\mathbf{P}_b$ is at most $k-1$, and the inverse of $\mathbf{P}$ is not defined. To ameliorate this, we consider the analysis computation to take place in the subspace S spanned by the columns of $\mathbf{X}_b$ (the ensemble states), and regard $\mathbf{X}_b$ as a linear transformation from a k-dimensional space $\tilde{S}$ into S. Letting $\mathbf{w}$ be a vector in $\tilde{S}$, then $\mathbf{X}_b\mathbf{w}$ belongs to S and $\mathbf{x} = \overline{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w}$. We can therefore write the cost function as

$$\begin{aligned} \tilde{J} =& \frac{1}{k-1}\mathbf{w}^T\mathbf{w} + (\mathbf{y}_o - H(\overline{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w}))^T \\ & \times \mathbf{R}^{-1} \, (\mathbf{y}_o - H(\overline{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w})) \end{aligned}$$

and it has been shown that this is an equivalent formulation by (Hunt et al. 2007).

Finally, we linearize H by applying it to each $\mathbf{x}_{b(i)}$ and interpolating. This is accomplished by defining

$$\mathbf{y}_{b(i)} = H(\mathbf{x}_{b(i)}) \tag{2.58}$$

such that

$$H(\overline{\mathbf{x}}_b - \mathbf{X}_b\mathbf{w}) = \overline{\mathbf{y}}_b - \mathbf{Y}_b\mathbf{w} \tag{2.59}$$

where $\mathbf{Y}$ is the $l \times k$ matrix whose columns are $\mathbf{y}_{b(i)} - \overline{\mathbf{y}}_b$ for $\overline{\mathbf{y}}_b$ the average of the $\mathbf{y}_{b(i)}$ over i. Now we rewrite the cost function for the linearize observation operator as

$$\begin{aligned}\tilde{J}^* =&(k-1)\mathbf{w}^T\mathbf{w} + (\mathbf{y}_o - \overline{\mathbf{y}}_b - \mathbf{Y}_b\mathbf{w})^T \\ &\times \mathbf{R}^{-1}(\mathbf{y}_o - \overline{\mathbf{y}}_b - \mathbf{Y}_b\mathbf{w})\end{aligned}$$

This cost function minimum is the form of the Kalman Filter as I have written it before:

$$\begin{aligned}\overline{\mathbf{w}}_a &= \tilde{\mathbf{P}}_a(\mathbf{Y}_b)^T\mathbf{R}^{-1}(\mathbf{y}_o - \overline{\mathbf{y}}_b), \\ \tilde{P}_a &= \left((k-1)\mathbf{I} + (\mathbf{Y}_b)^T\mathbf{R}^{-1}\mathbf{Y}_b\right)^{-1}.\end{aligned}$$

In model space these are

$$\begin{aligned}\tilde{\mathbf{x}}_a &= \overline{\mathbf{x}}_b + \mathbf{X}_b\overline{\mathbf{w}}_a \\ \mathbf{P}_a &= \mathbf{X}_b\tilde{\mathbf{P}}_a(\mathbf{X}_b)^T.\end{aligned}$$

We move directly to a detailed description of the computational implementation of this filter. Starting with a collection of background forecast vectors $\{\mathbf{x}_{b(i)} : i = 1..k\}$, we perform steps 1 and 2 in a global variable space, then steps 3-8 for each grid point:

1. apply H to $\mathbf{x}_{b(i)}$ to form $\mathbf{y}_{b(i)}$, average the $\mathbf{y}_b$ for $\overline{\mathbf{y}_b}$, and form $\mathbf{Y}b$.

2. similarly form $\mathbf{X}_b$. (now for each grid point)

3. form the local vectors

4. compute $\mathbf{C} = (\mathbf{Y}_b)^T \mathbf{R}^{-1}$ (perhaps by solving $\mathbf{R}\mathbf{C}^T = \mathbf{Y}_b$

5. compute $\tilde{\mathbf{P}}_a = ((k-1)\mathbf{I}/\rho + \mathbf{C}\mathbf{Y}_b)^{-1}$ where $\rho > 1$ is a tunable covariance inflation factor

6. compute $\mathbf{W}_a = \left((k-1)\tilde{\mathbf{P}}_a\right)^{1/2}$

7. compute $\overline{\mathbf{w}}_a = \tilde{\mathbf{P}}_a \mathbf{C} \left(\mathbf{y}_o - \overline{\mathbf{y}}_b\right)$ and add it to the column of $\mathbf{W}_a$

8. multiply $\mathbf{X}_b$ by each $\mathbf{w}_{a(i)}$ and add $\tilde{\mathbf{x}}_b$ to get $\left\{\mathbf{x}_{a(i)} : i = 1..k\right\}$ to complete each grid point

9. combine all of the local analysis into the global analysis

The greatest implementation difficulty in the OpenFOAM model comes back to the spatial discretization and defining the local vectors for each grid point.

Four Dimensional Local Ensemble Transform Kalman Filter (4D-LETKF)

We expect that extending the LETKF to incorporate observations from within the assimilation time window will become useful in the context of our target experiment high-frequency temperature sensors. This is implemented without great difficulty on top of the LETKF, by projecting observations as they are made into the space $\tilde{S}$ with the observation time-specific matrix H_τ and concatenating each successive observation onto $\mathbf{y}_o$. Then with the background forecast at the time τ, we compute $(\overline{\mathbf{y}}_b)_\tau$ and $(\mathbf{Y}_b)_\tau$, concatenating onto $\overline{\mathbf{y}}_b$ and $\mathbf{Y}_b$, respectively.

2.5 4D-Var

The formulation of 4D-Var aims to incorporate observations at the time they are observed. This inclusion of observations has resulted in significant improvements in forecast skill, cite. The next generation of supercomputers that will be used in numerical weather prediction will allow for increased model resolution. The main issues with the 4D-Var approach include attempting to parallelize what is otherwise a sequential algorithm, and the computational difficulty of the strong-constraint formulation with many TLM integrations, with work being done by Fisher and Andersson (2001).

The formulation of 4D-Var is not so different from 3D-Var, with the inclusion of a TLM and adjoint model to assimilation observations made within the assimilation window. At the ECMWF, 4D-Var is run on a long-window cycle (24 and 48 hours) so that many observations are made within the time window. Generally, there are two main 4D-Var formulations in use: strong-constraint and weak-constaint. The strong-constraint formulation uses the forecast model as a strong constraint, and is therefore significantly easier to solve. From Kalnay (2007) we have the standard weak-constaint formulation with superscript denoting the timestep within the assimilation window (say there are $0, \ldots, N$ observation time steps within the assimilation window):

$$J(\mathbf{x}^0) = \frac{1}{2}(\mathbf{x}^0 - \mathbf{x}_b^0)^T \mathbf{B}^{-1} (\mathbf{x}^0 - \mathbf{x}_b^0 \frac{1}{2} \sum_{i=0}^{N} \left(H^i(\mathbf{x}^i) - \mathbf{y}^i \right)^T (\mathbf{R}^i)^{-1} \left(H^i(\mathbf{x}^i) - \mathbf{y}^i \right).$$

The weak-constraint formulation has only recently been implemented operationally. This is (Kalnay et al. 2007):

$$
\begin{aligned}
J(\mathbf{x}^0, \beta) =& \frac{1}{2}(\mathbf{x}^0 - \mathbf{x}_b^0)^T \mathbf{B}^{-1} (\mathbf{x}^0 - \mathbf{x}_b^0) + \frac{1}{2} \sum_{i=1}^{N} \\
& \left(H^i(\mathbf{x}^i + \beta) - \mathbf{y}^i\right)^T (\mathbf{R}^i)^{-1} \left(H^i(\mathbf{x}^i + \beta) - \mathbf{y}^i\right) \\
& + \frac{1}{2}\beta^T \mathbf{Q}^{-1} \beta = J_b + J_o + J_\beta.
\end{aligned}
$$

Future directions of operational data assimilation combine what is best about 4D-Var and ensemble-based algorithms, and these methods are reffered to as ensemble-variational hybrids. We have already discussed a few such related approaches: the use of the Kalman gain matrix $\mathbf{K}$ used within the 3D-Var cost function, the “NCEP” method for 3D-Var, and inflating the ETKF covariance with the 3D-Var static (climatological) covariance $\mathbf{B}$.

2.5.1 Ensemble-variational hybrids

Ensemble-variational hybrid approaches benefit from flow dependent covariance estimates obtained from an ensemble method, and the computational efficiency of iteratively solving a cost function to obtain the analysis, are combined. An ensemble of forecasts is used to update the static error covariance $\mathbf{B}$ in the formulation of 4D-Var (this could be done in 3D-Var as well) with some weighting from the climatological covariance $\mathbf{B}$ and the ensemble covariance. This amounts to a formulation of

$$
\mathbf{B} = \alpha \mathbf{P}_f + (1 - \alpha)\mathbf{B} \tag{2.60}
$$

for some α between 0 and 1. Combining the static error covariance $\mathbf{B}$ and the flow dependent error covariance $\mathbf{P}_f$ is an active area of research (Clayton et al. 2012).

Chapter 3

Computational Fluid Dynamics

In this chapter I present the basics of CFD that will be necessary to understand our approach to modeling the thermosyphon numerically. This consists of a presentation of the equations, and a discussion of their solution on a finite grid.

3.1 Governing Equations

We consider the incompressible Navier-Stokes equations with the Boussinesq approximation to model the flow of water inside a thermal convection loop. Here we present the main eqauations that are solved numerically, noting the assumptions that were necessary in their derivation. In Appendix B a more complete derivation of the equations governing the fluid flow, and the numerical implementation, for our problem is presented. In standard notation, the continuity equation for an incompressible fluid is

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = 0. \tag{3.1}$$

The following three equations, presented compactly in tensor notation, are given by

$$g_i \frac{\bar{\rho}}{\rho_0} = g_i \left(1 + \frac{\bar{\rho} - \rho_0}{\rho_0}\right) = g_i \rho_k = g_i \left(1 - \beta \left(\overline{T} - T_0\right)\right).$$

Note that $g_i = 0$ for $i \in \{x, y\}$ since gravity is assumed to be the z direction. The energy equation is given by

$$\frac{\partial}{\partial t} (\rho \bar{e}) + \frac{\partial}{\partial x_j} (\rho \overline{e u}_j) = -\frac{\partial q_k^*}{\partial x_k} - \frac{\partial \overline{q}_k}{\partial x_k}. \tag{3.2}$$

3.2 Discretization and Solving Algorithms

A spatial and temporal discretization of the governing equations is necessary so that they may be solved numerically. In general, there are three approaches to the numerical solution of PDEs: finite differences, finite elements, and finite volumes. OpenFOAM has the capacity for each scheme, and for our problem with an unstructed mesh, the finite volume method is the simplest.

After discretization, solving is accomplished by the Pressure-Implicit Split Operator (PISO) algorithm (Issa 1986). In Table 3.1, we note the choices of schemes for time-stepping, numerical methods and interpolation.

3.2.1 Solving with OpenFOAM

Using the aforementioned numerical schemes, we perform a computational fluid dynamics simulation of the thermosyphon using the open-source software OpenFOAM. The solver in OpenFOAM that I use, with some modification, is “buoyantBoussinesqPimpleFoam.” Modification of the code was necessary for laminar operation.

The Boussinesq approximation makes the assumption that density changes due to temperature differences are sufficiently small so that they can be neglected without effecting the flow, except when multiplied by gravity. The non-dimensionalized density difference $\Delta\rho/\rho$ depends on the value of heat flux into the loop, but remains small even for large values of flux.

$$\frac{\beta(T - T_{\text{ref}})}{\rho_{\text{ref}}} = 7.6 \cdot 10^{-6} \ll 1 \tag{3.3}$$

Available boundary conditions (BCs) that were found to be stable in OpenFOAM's solver were constant gradient, fixed value conditions, and turbulent heat flux. Simulations with a fixed flux BC is implemented through the externalWallHeatFluxTemperature library were unstable and resulted in physically unrealistic results. Constant gradient simulations were stable, but the behavior was empirically different from our physical system. Results from attempts at implementing these various boundary conditions are presented in Table 4.4.2. To most accurately model our experiment, we choose a fixed value BC. This is due to the thermal diffusivity and thickness of the walls of the experimental setup. ☐ Derive condition

Problem	Scheme
Main solver	buoyantBoussinesqPimpleFoam
Solving algorithm	PISO
Grid	Semi-staggered
Turbulence model	None (laminar)
Time stepping	Euler
Gradient	Gauss linear
Divergence	Gauss upwind
Laplacian	Gauss linear
Interpolation	Linear
Matrix preconditioner	Diagonal LU
Matrix solver	Preconditioned Bi-linear Conjugate Gradient

Table 3.1: Numerical schemes used for solving the finite volume problem.

Variable	Value (at 300K, if applicable)
Rayleigh number	$[1.5 \cdot 10^6, 4 \cdot 10^7]$
Reynolds number	≈ 416
Gravity g_z (fixed)	9.8m/s^2
Specific heat c_p	4.187Kj/Kg
Thermal expansion β	$0.303 \cdot 10^{-3}\, 1/\text{K}$
Density ρ_{ref}	995.65Kg/m^3
Kinematic Viscoscity ν	$0.801 \cdot 10^{-6}\text{m}^2/\text{s}$
Dynamic Viscoscity μ	0.798
Laminar Prandtl number	5.43
Boussinesq condition	$7.6 \cdot 10^{-6}$

Table 3.2: Specific constants used for thermal properties of water. For temperature dependent quantities, the value is evaluated at the reference temperature $T_{\text{ref}} = 300K$. Derivation of the Boussineq condition is presented in Equation 3.3.

Chapter 4

Thermal Convection Loop Experiment

> The thermosyphon, a type of natural convection loop or non-mechanical heat pump, can be likened to a toy model of climate (Harris et al. 2011).

4.1 Thermal Convection Loops

Under certain conditions, thermal convection loops operate as non-mechanical heat pumps, making them useful in many applications. In Figure 4.1 we see three such applications. In solar hot water heaters, water is warmed using the sun's energy, resulting in a substantial energy savings in solar-rich areas. Solar hot water thermosyphons, filled with either water or antifreeze, are used to transfer the solar energy into a hot water tank. Geothermal systems rely on the Earth's warmth deep underground to heat and cool buildings, and due to low temperature differences are often augmented with a physical pump. Cooling the CPU of modern computers is necessary for operation at high speeds, and thermosyphons have been used here to reduce both noise and energy consumption. The author notes the difference between forced and natural convection, where in some of these systems forced convection may be necessary in operation.

In response to a warming climate, thermosyphons have been used to transfer heat away from buildings built on fragile permafrost, which would otherwise melt and destroy the structure (Xu and Goering 2008). These ground thermosyphons are pictured in Figure 4.2.

Figure 4.1: Here we see many different thermosyphon applications. Clockwise starting at the top, we have a geothermal heat exchanger, solar hot water heater, and computer cooler. Images credit to BTF Solar and Kuemel Bernhard (Bernhard 2005, Carlton 2009).

4.2 Ehrhard-Müller Equations

Following the derivation by Harris (Harris et al. 2011), itself a representation of the derivation of Gorman (Gorman et al. 1986) and namesakes Ehrhard and Müller (Ehrhard and Müller 1990), we derive the equations governing a closed loop thermosyphon.

Similar to the derivation of the governing equations of CFD in Chapter 3, I start with a small but finite volume inside the loop. Here, however, the volume is described by $\pi r^2 R \mathrm{d}\phi$ for r the interior loop size (such that πr^2 is the area of a slice) and $R\mathrm{d}\phi$ the arc length (width) of the slice. Newton's second law states that momentum is conserved, such that the sum of the forces acting upon our finite volume is equal to the change in momentum of this

Figure 4.2: Thermosyphons flank a Federal Aviation Administration building in Fairbanks, Alaska. The devices help keep the permafrost frozen beneath buildings and infrastructure by transferring heat out of the ground. Photo credit Jim Carlton (Carlton 2009).

volume. Therefore we have the basic starting point for forces $\sum F$ and velocity u as

$$\sum F = \rho \pi r^2 R \mathrm{d}\phi \frac{\mathrm{d}u}{\mathrm{d}t}. \tag{4.1}$$

The sum of the forces is $\sum F = F_{\{p,f,g\}}$ for net pressure, fluid shear, and gravity, respectively. We write these as

$$F_p = -\pi r^2 \mathrm{d}\phi \frac{\partial p}{\partial \phi} \tag{4.2}$$

$$F_w = -\rho \pi r^2 \mathrm{d}\phi f_w \tag{4.3}$$

$$F_g = -\rho \pi r^2 \mathrm{d}\phi g \sin(\phi) \tag{4.4}$$

where $\partial p / \partial \phi$ is the pressure gradient, f_w is the wall friction force, and $g \sin(\phi)$ is the vertical component of gravity acting on the volume. As in the derivation of the governing equations, we now introduce the Boussinesq approximation which states that both variations in fluid density are linear in temperature T and density variation is insignificant except when multiplied by gravity. The consideration manifests as

$$\rho = \rho(T) \simeq \rho_{\text{ref}}(1 - \gamma(T - T_{\text{ref}})$$

where ρ_0 is the reference density and T_{ref} is the reference temperature. The second consideration of the Boussinesq approximation allows us to replace ρ with this ρ_{ref} in all terms except for F_g. We now write momentum equation as

$$-\pi r^2 \mathrm{d}\phi \frac{\partial p}{\partial \phi} - \rho_{\text{ref}} \phi r^2 R \mathrm{d}\phi f_w - \rho_{\text{ref}}(1 - \rho(T - T_{\text{ref}}))\pi r^2 R \mathrm{d}\phi g \sin(\phi) = \rho_{\text{ref}} \pi r^2 R \mathrm{d}\phi \frac{\mathrm{d}u}{\mathrm{d}t}. \tag{4.5}$$

Canceling the common πr^2, dividing by R, and pulling out dϕ on the LHS we have

$$-\mathrm{d}\phi\left(\frac{\partial p}{\partial \phi}\frac{1}{R} - \rho_{\text{ref}} f_w - \rho_{\text{ref}}(1 - \rho(T - T_{\text{ref}}))g\sin(\phi)\right) = \rho_{\text{ref}}\mathrm{d}\phi\frac{\mathrm{d}u}{\mathrm{d}t}. \tag{4.6}$$

We integrate this equation over ϕ to eliminate many of the terms, specifically we have

$$\begin{aligned}
&\int_0^{2\pi} -\mathrm{d}\phi\frac{\partial p}{\partial \phi}\frac{1}{R} \to 0 \\
&\int_0^{2\pi} -\mathrm{d}\phi\rho_{\text{ref}} g\sin(\phi) \to 0 \\
&\int_0^{2\pi} -\mathrm{d}\phi\rho_{\text{ref}}\gamma T_{\text{ref}} g\sin(\phi) \to 0.
\end{aligned}$$

Since u (and hence $\frac{\mathrm{d}u}{\mathrm{d}\phi}$) and f_w do not depend on ϕ, we can pull these outside an integral over ϕ and therefore the momentum equation is now

$$2\pi f_w\rho_0 + \int_0^{2\pi} \mathrm{d}\phi\rho_{\text{ref}}\gamma T g\sin(\theta) = 2\pi\frac{\mathrm{d}u}{\mathrm{d}\phi}\rho_{\text{ref}}.$$

Diving out 2π and pull constants out of the integral we have our final form of the momentum equation

$$f_w\rho_{\text{ref}} + \frac{\rho_{\text{ref}}\gamma g}{2\pi}\int_0^{2\pi} \mathrm{d}\phi T\sin(\theta) = \frac{\mathrm{d}u}{\mathrm{d}\phi}\rho_{\text{ref}}. \tag{4.7}$$

Now considering the conservation of energy within the thermosyphon, the energy change within a finite volume must be balanced by transfer within the thermosyphon and to the walls. The internal energy change is given by

$$\rho_{\text{ref}}\pi r^2 R\mathrm{d}\phi\left(\frac{\partial T}{\partial t} + \frac{u}{R}\frac{\partial T}{\partial \phi}\right) \tag{4.8}$$

which must equal the energy transfer through the wall

$$\dot{q} = -\pi r^2 R \mathrm{d}\phi h_w (T - T_w). \tag{4.9}$$

Combining Equations 4.8 and 4.9 (and canceling terms) we have the energy equation:

$$\left(\frac{\partial T}{\partial t} + \frac{u}{R} \frac{\partial T}{\partial \phi} \right) = \frac{-h_w}{\rho_{\mathrm{ref}} c_p} \left(T - T_w \right). \tag{4.10}$$

The f_w which we have yet to define and h_w are fluid-wall coefficients and can be described by (Ehrhard and Müller 1990):

$$\begin{aligned} h_w &= h_{w_0} \left(1 + K h(|x_1|) \right) \\ f_w &= \frac{1}{2} \rho_{\mathrm{ref}} f_{w_0} u. \end{aligned}$$

We have introduced an additional function h to describe the behavior of the dimensionless velocity $x_1 \alpha u$. This function is defined piece-wise as

$$h(x) = \begin{cases} x^{1/3} & \text{when } x \geq 1 \\ p(x) & \text{when } x < 1 \end{cases}$$

where $p(x)$ can be defined as $p(x) = \left(44x^2 - 55x^3 + 20x^4\right)/9$ such that p is analytic at 0 (Harris et al. 2011).

Taking as an approximation solution the lowest modes of a Fourier expansion for T we consider

$$T(\phi, t) = C_0(t) + S(t)\sin(\phi) + C(t)\cos(\phi) \tag{4.11}$$

by substituting this form into Equations 4.7 and 4.10 and integrating. We then follow the particular nondimensionalization choice of Harris et al such hat we the following ODE system, which we refer to as the Ehrhard-Müller equations:

$$\frac{\mathrm{d}x_1}{\mathrm{d}t'} = \alpha(x_2 - x_1) \tag{4.12}$$
$$\frac{\mathrm{d}x_2}{\mathrm{d}t'} = \beta x_1 - x_2(1 + Kh(|x_1|)) - x_1 x_3 \tag{4.13}$$
$$\frac{\mathrm{d}x_3}{\mathrm{d}t'} = x_1 x_2 - x_3(1 + Kh(|x_1|)) - x_1 x_3. \tag{4.14}$$

The nondimensionalization is given by the change of variables

$$t' = \frac{h_{w_0}}{\rho_{\text{ref}} c_p} t \tag{4.15}$$
$$x_1 = \frac{\rho_{\text{ref}} c_p}{R h_{w_0}} u \tag{4.16}$$
$$x_2 = \frac{1}{2} \frac{\rho_{\text{ref}} c_p \gamma g}{R h_{w_0} f_{w_0}} \Delta T_{3-9} \tag{4.17}$$
$$x_3 = \frac{1}{2} \frac{\rho_{\text{ref}} c_p \gamma g}{R h_{w_0} f_{w_0}} \left(\frac{4}{\pi} \Delta T_w - \Delta T_{6-12} \right) \tag{4.18}$$

and

$$\alpha = \frac{1}{2} R c_p f_{w_0} / h_{w_0} \tag{4.19}$$
$$\beta = \frac{2}{\pi} \frac{\rho_{\text{ref}} c_p \gamma g}{R h_{w_0} f_{w_0}} \Delta T_w. \tag{4.20}$$

Through careful consideration of these non-dimensional variable transformations we verify that x_1 is representative of the mean fluid velocity, x_2 of the temperature difference between the 3 and 9 o'clock positions on the thermosyphon, and x_3 the deviation from the vertical temperature profile in a conduction state (Harris et al. 2011).

4.3 Detailed Experimental Setup

Originally built (and subsequently melted) by Dr. Chris Danforth at Bates College, the University of Vermont's current experimental thermosyphons have made considerable progress. Operated by Dave Hammond, UVM's Scientific Electronics Technician, the experimental thermosyphons access the chaotic regime of state space found in the principled governing equations.

4.4 Computational Experiment

Using the aforementioned CFD techniques and modified solver "buoyantBoussinesqPimpleFoam" we simulate the thermosyphon. First, we begin by creating 2 and 3-dimensional meshes of the thermosyphon natively in OpenFOAM's blockMesh utility. Then we specify boundary conditions, searching over many possible use cases. Finally, I discuss some final considerations of our model, including the finite volume schemes and turbulence modeling.

Here is a picture of the thermosyphon colored by temperature.

4.4.1 Mesh

Creating a suitable mesh is paramount to making an accurate simulation. To reduce computational bandwidth, it was important to first create the mesh and then renumber using the renumberMesh utility. The 2D mesh contains 40,382 points, and is imposed with fixed value boundary conditions on the top and bottom of the loop.

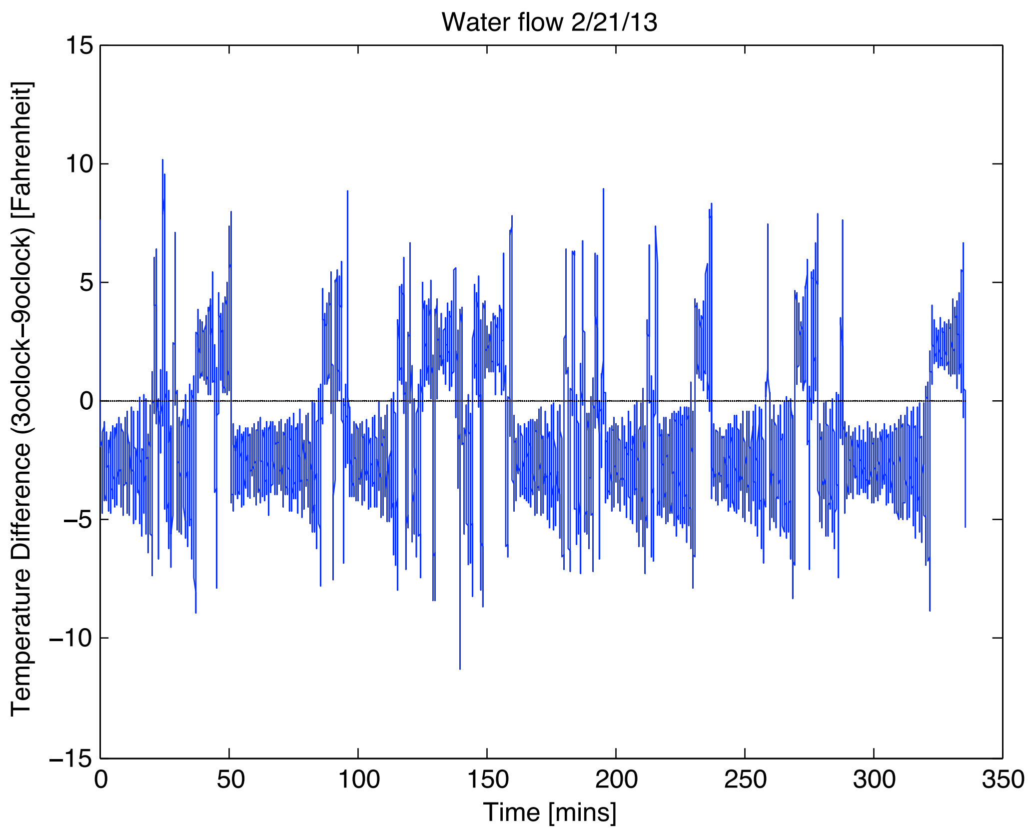

Figure 4.3: A time series of the physical thermosyphon, from the Undergraduate Honor's Thesis of Darcy Glenn (Glenn 2013). The temperature difference (plotted) is taken as the difference between temperature sensors in the 3 and 9 o'clock positions. The sign of the temperature difference indicates the flow direction, where positive values are clockwise flow.

4.4.2 Boundary conditions

There are a wide variety of thermophysical boundary conditions available. In Table 4.4.2 we list the boundaries conditions with which we experimented, and the quality of the resulting simulation.

Figure 4.4: Professor Chris Danforth with the Thermal Convection Loop. Photo credit Tim Johnson, Burlington Free Press (Johnson 2009).

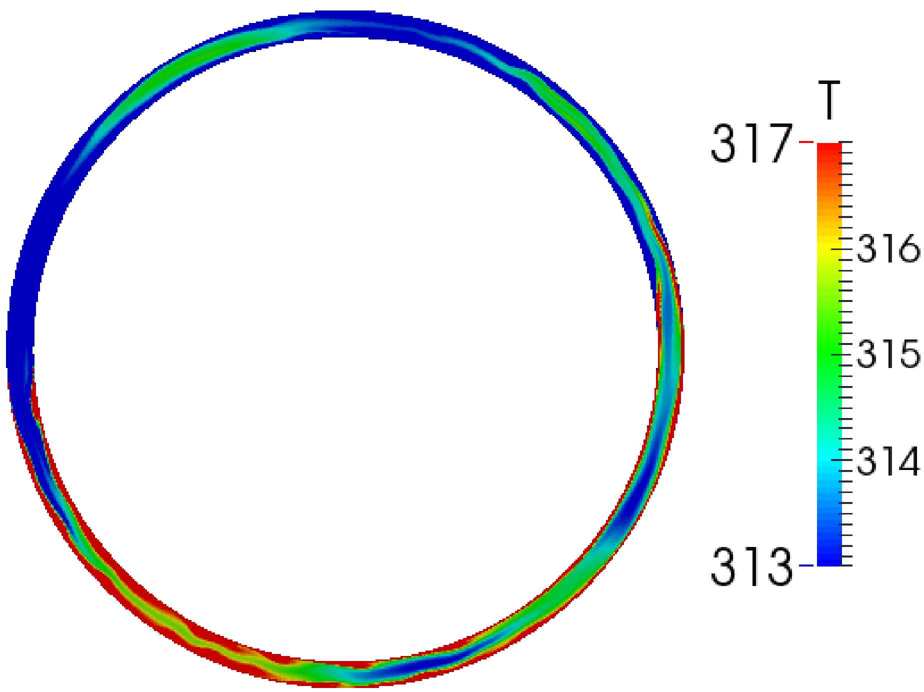

Figure 4.5: A screenshot of the whole loop. Shown is an initial stage of heating for a fixed value boundary condition, 2D, laminar simulation with a mesh of 40000 cells including wall refinement with walls heated at 340K on the bottom half and cooled to 290K on the top half.

4.4.3 Other considerations

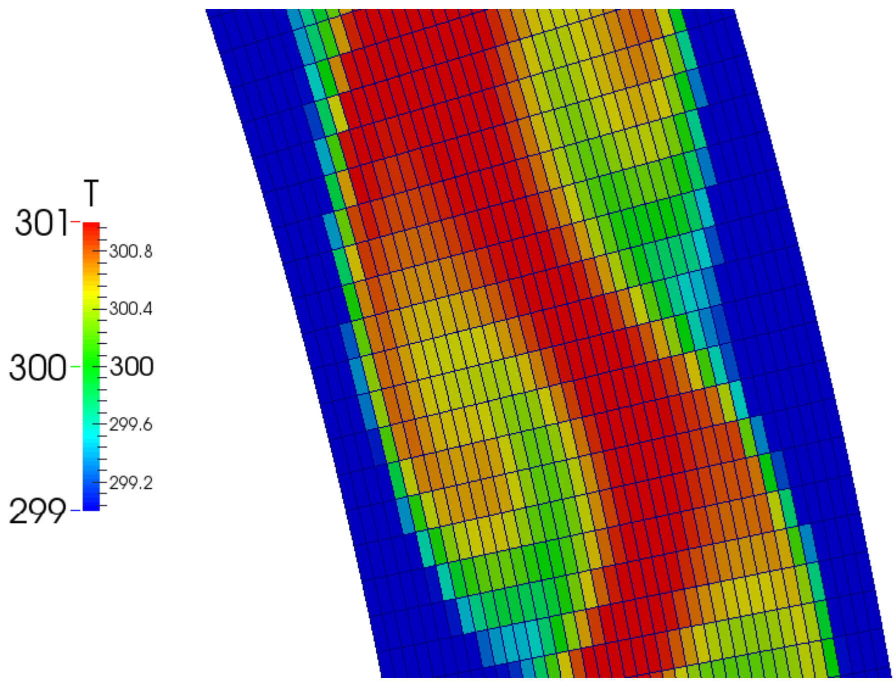

Figure 4.6: A snapshot of the mesh used for CFD simulations. Shown is an initial stage of heating for a fixed value boundary condition, 2D, laminar simulation with a mesh of 40000 cells including wall refinement with walls heated at 340K on the bottom half and cooled to 290K on the top half.

Boundary Condition	Type	Values Considered	Result
fixedValue	value	$(270 \to 310) \times (300 \to 350)$	good
fixedGradient	gradient	$\pm 10 \to \pm 100000$	okay
wallHeatTransfer	enthalpy	-	won't run
externalWallHeatFluxTemperature	flux	-	won't run
turbulentHeatFluxTemperature	flux	$\pm .001 \to \pm 10$	terrible

Table 4.1: Boundary conditions considered for the OpenFOAM CFD model are labeled with their type, values considered, and empirical result. We find the most physically realistic simulations using the simplest "fixedValue" condition. The most complex conditions, "wallHeatTransfer," "externalWallHeatFluxTemperature," and "turbulentHeatFluxTemperature" either did not compile or produced divergent results.

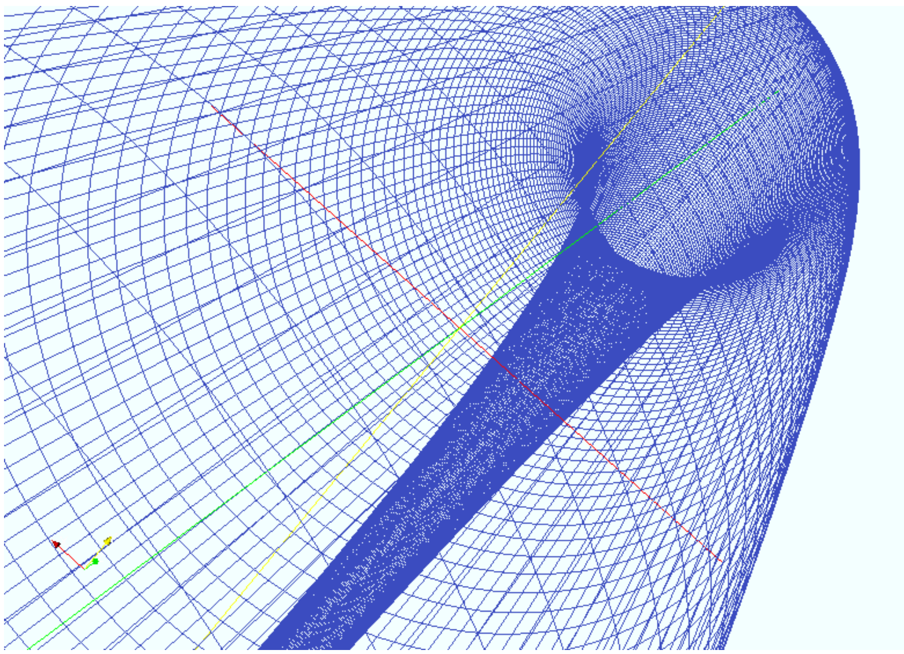

Figure 4.7: The 3D mesh viewed as a wireframe from within. Here there are 900 cells in each slice (not shown), for a total mesh size of 81,000 cells. Simulations using this computational mesh are prohibitively expensive for use in a real time ensemble forecasting system, but are possible offline.

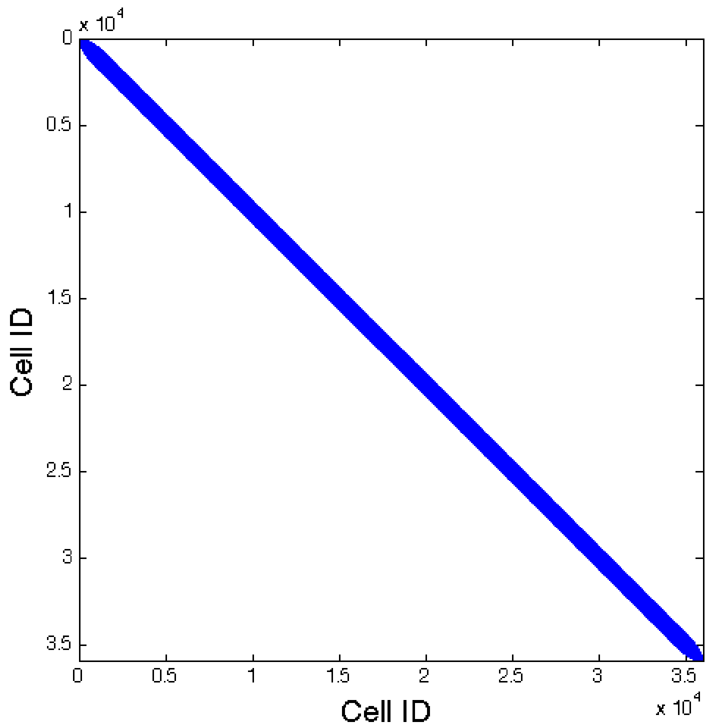

Figure 4.8: The 2D mesh Adjacency matrix for a local radius of 1.0cm. In general, we see that cells have neighbors with nearby cell ID. With 5,260,080 nonzero entries, it is difficult to see structure looking at the entire matrix.

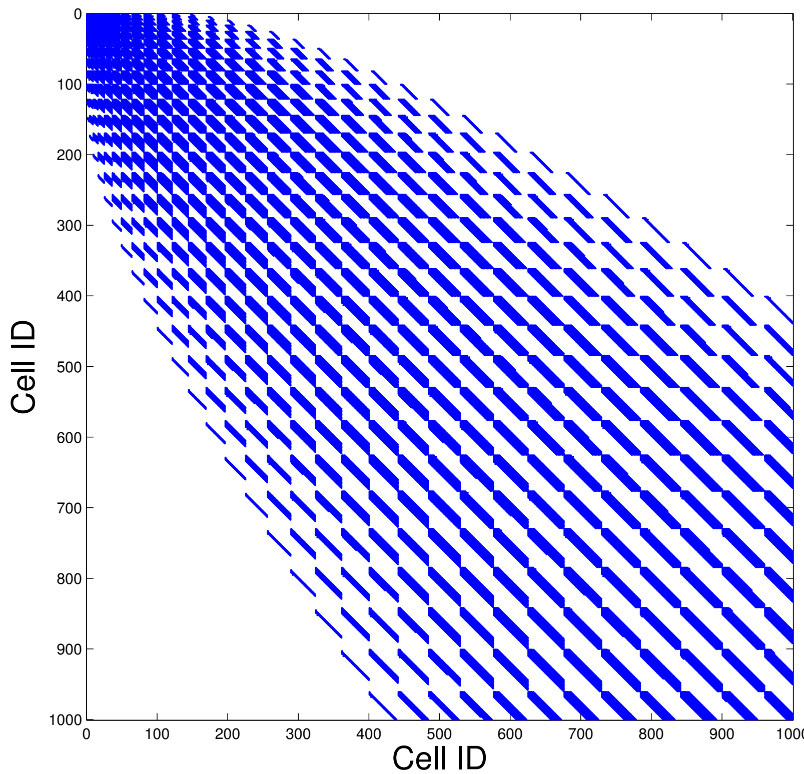

Figure 4.9: The 2D mesh Adjacency matrix, zoomed in. The Cuthill-McKee renumbering algorithm was used, and we see 23 distinct regions of local numbering, reflective of the geometry and Cuthill-McKee algorithm. The mesh shown here has 30 cells in each perpendicular slice of the loop, and they are numbered iteratively along the edge of triangular front emanating from cell 0 (chose to be the left-most cell). This diagonalization in both space and ID number effectively reduces the computational bandwidth of the mesh from 947 to 18, as computed by the "renumberMesh" utility. □ define bandwidth

Chapter 5

Results

5.1 Computational Model and DA Framework

5.1.1 Core framework

The first output of this work is a general data assimilation framework for MATLAB. By utilizing an object-oriented (OO) design, the model and data assimilation algorithm code are separate and can be changed independently. The principal advantage of this approach is the ease of incorporation of new models and DA techniques.

To add a new model to this framework, it is only necessary to write a MATLAB class wrapper for the model, and the existing Lorenz 63 and OpenFOAM model wrappers can be used as templates. Testing a new technique for performing the DA step is as simple as writing a MATLAB function for the assimilation steps, where the the model result and observations are provided as an input.

The core of this framework is the MATLAB script modelDAinterface.m which wraps observations, the model class, and any DA method together. A description of the main algorithm is provided in the Appendix. ☐ remove this statement if the code can't be included succinctly

For large models, it is necessary to perform the assimilation in a local space, and this consideration is accounted for within the framework.

5.1.2 Tuned OpenFOAM model

In addition to wrapping the OpenFOAM model into MATLAB-speak, defining a physically realistic simulation was necessary for accurate results. A non-uniform sampling of the OpenFOAM model parameter space across BC type, BC values, and turbulence model was performed. The amount of output data constitutes what is today "big data," and for this reason it was difficult to find a suitable parameter regime systematically. Specically, each model run generates 5MB of data per time save, and with 10000 timesteps this amounts to roughly 50GB of data for each model run. I then chose the turbulence model, BC type, and BC values based on a empirical selection of the time-series for a subset of model variables that best represented the known chaotic regime of a physical model.

5.2 Prediction Skill for Synthetic Data

I first present the results pertaining to the accuracy of forecasts for synthetic data. There are many possible experiments given the choice of assimilation window, data assimilation algorithm, localization scheme, model resolution, observational density, observed variables, and observation quality. I focus on considering the effect of observations, assimilation window, and covariance localization scheme on the resulting forecast skill.

Synthetic data is generated from one model run, pertaining a modified "buoyantBoussinesqPimpleFoam" solution with fixed value boundary conditions of 290K top and 340K bottom, 80,000 grid points, a time step of .01, and laminar turbulence model. Data is saved for every one second, and is subsampled for 2,4,8,16 and 32 synthetic temperature sensors spaced evenly around the loop. The temperature is reported as the average for cells with centers within 0.5cm of the observation center. In addition to temperature sensing, the velocity in y and z is sampled at 50, 100, and 300 points chosen randomly at each time step. These velocity observations, when included are designed to simulate an reconstructed velocity measurement based on video particle tracking. To obtain realistic sampling intervals, this data was aggregated and is then subsampled at the assimilation window length.

Errors for both the EM model and OpenFOAM are presented as RMSE from the observed T_{3-9}, such that the models can be compared. In the EM model, a maximum of 2 temperature sensors are considered for observations at T_3 and T_9, and no mean velocity reconstruction from subsampled velocity is attempted.

For the predictions to work, it was first necessary to scale the model variables to the same order of magnitude. This is accomplished for these results by subtracting the mean climatological values from the observations and model results before assimilation. While other, more complex, strategies may result in better performance there are no general rules to determine the best scaling factors and good scaling is often problem dependent (Navon et al. 1992, Mitchell 2012).

The need for covariance localization was discussed in Chapter 2, and here we implement three different schemes. First, we consider cells within each perpendicular slice of the loop to be a cohesive group, and their neighbors to those cells within r slices (where here,

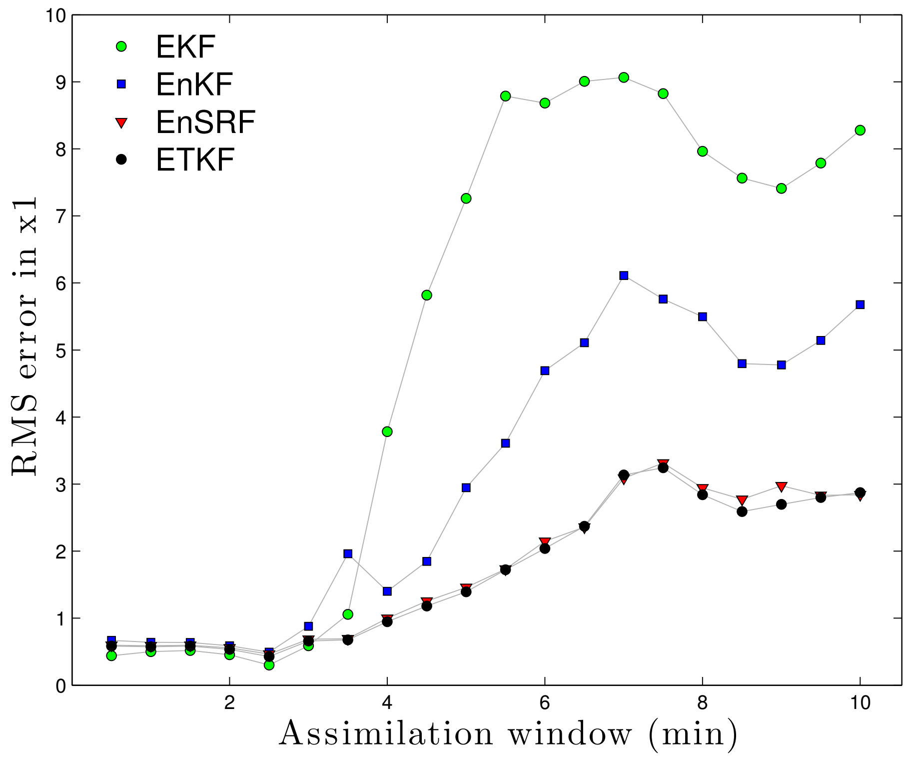

Figure 5.1: A first pass (covariance inflation not tuned) experiment of Lorenz 63 (EM) prediction skill vs increasing assimilation windows on real data. Notice that the error explodes at an window length near the typical time length of flow reversals.

r is an integer). Computationally this scheme is the simplest to implement, and results in the fewest covariance localizations.

Second to the plate is localization by distance from center. Data assimilation is performed for each grid point, considering neighboring cells and observations with cell center within a radius r from each point (e.g. $r = 0.5$cm).

And finally, we attempt a novel adaptive covariance localization scheme based on the identifying coherent flow structures. Flow structures are identified by considering the leading vectors in U of the singular value decomposition (SVD) of flow within a reasonably sized domain (e.g. within slice localization regions for large r). This corresponds to a

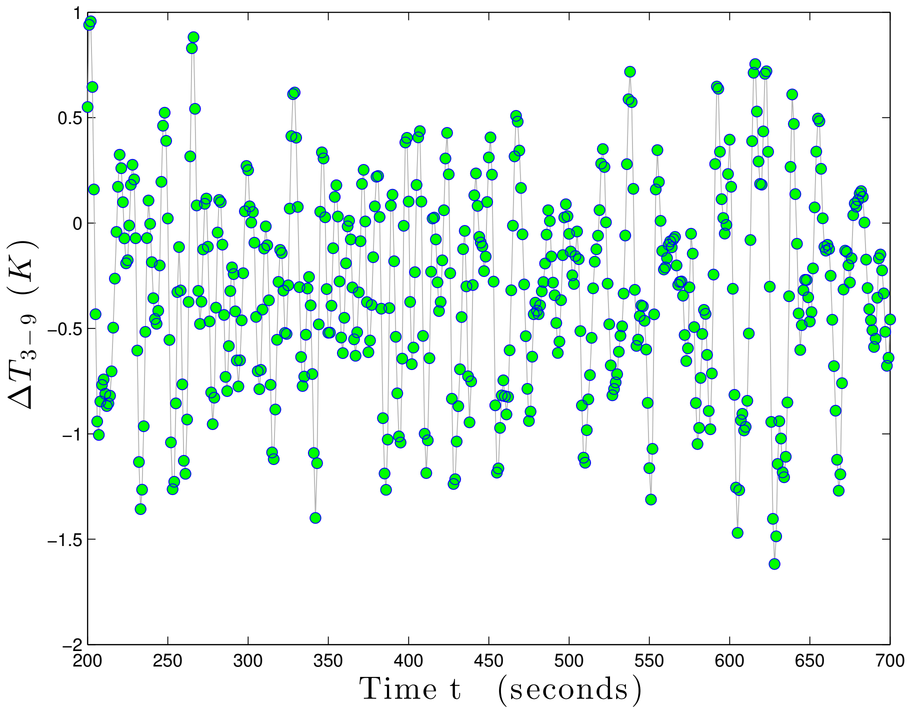

Figure 5.2: The CFD generated timeseries used as synthetic data to test predictions.

proper orthogonal decomposition (POD) of the flow structures. □ finalize, relate to current methods, properly describe the math....this is still in testing

5.3 Prediction Skill for Real Data

□ obtain a physical time series, input into previous synthetic experiment! I think that this is possible, if not in the next week before my oral presentation, in the next three before this document goes to press (and archival). All the tools are in place and running on synthetic data, so all it needs is the timeseries.

5.4 Conclusion

Throughout the process of building a data assimilation scheme from scratch, as well as deriving and tuning a computational model, I have learned a great deal about the difficulties and successes of modeling. I have found that data assimilation algorithms and CFD solvers are sensitive to parameter choices, and cannot be used successfully without an approach that is specific to the problem at hand.

For the future of data assimilation, the numerical schemes will need to be adaptive to the new computing resources. The trend toward massively parallel computation puts the spotlight on effective localization of the assimilation step, which may be difficult in schemes that do not use ensembles, such as the 4D-Var. Maintaining TLM and adjoint code is difficult and time-consuming, and with future increases in model resolution and atmospheric understanding, TLMs of full GCM models may become impossible. Again, this points to the potential use of local ensemble based filters such as the LETKF as implemented here.

5.4.1 Future directions

The immediate next step for this work is to test prediction skill in real time. All of the pieces are there: an observation-based data assimilation framework and models that run at near real-time speed. Currently on the VACC, the OpenFOAM simulation with grid size 36K runs with approximately half of real-time speed on one core. This allows the possibility of running N ensemble methods using $8N$ cores.

Beyond real-time prediction, it is now possible to design and test new methods of data assimilation for large scale systems. New methods specific to CFD, possibly using principle mode decomposition (PMD) of flow fields have the potential to greatly improve the skill of real-time CFD applications.

The numerical coupling of CFD to experiment by DA should be generally useful to improve the skill of CFD predictions in even data-poor experiments, which can provide better knowledge of unobservable quantities of interest in fluid flow.

In data we trust.

Bibliography

Achtelik, M., T. Zhang, K. Kuhnlenz, and M. Buss (2009). Visual tracking and control of a quadcopter using a stereo camera system and inertial sensors. In *Mechatronics and Automation, 2009. ICMA 2009. International Conference on*, pp. 2863–2869. IEEE.

Allgaier, N. A., K. D. Harris, and C. M. Danforth (2012). Empirical correction of a toy climate model. *Physical Review E 85*(2), 026201.

Anderson, J. D. et al. (1995). *Computational fluid dynamics*, Volume 206. McGraw-Hill New York.

Anderson, M. J. (2001). A new method for non-parametric multivariate analysis of variance. *Austral Ecology 26*(1), 32–46.

Anderson Jr, J. (2009). Governing equations of fluid dynamics. In *Computational fluid dynamics*, pp. 15–51. Springer.

Asur, S. and B. A. Huberman (2010). Predicting the future with social media. In *Web Intelligence and Intelligent Agent Technology (WI-IAT), 2010 IEEE/WIC/ACM International Conference on*, Volume 1, pp. 492–499. IEEE.

Beine, B., V. Kaminski, and W. Von Lensa (1992). Integrated design of prestressed cast-iron pressure vessel and passive heat removal system for the reactor cell of a 200 mwth modular reactor. *Nuclear engineering and design 136*(1), 135–141.

Beitelmal, M. H. and C. D. Patel (2002). Two-phase loop: Compact thermosyphon. *Approved for External Publication*.

Belessiotis, V. and E. Mathioulakis (2002). Analytical approach of thermosyphon solar domestic hot water system performance. *Solar Energy 72*(4), 307–315.

Bernhard, K. (2005, November). Cpu vapor cooling thermosyphon. `http://www.overclockers.com/cpu-vapor-cooling-thermosyphon/`.

Bishop, C. H., B. J. Etherton, and S. J. Majumdar (2001). Adaptive sampling with the ensemble transform kalman filter. part i: Theoretical aspects. *Monthly weather review 129*(3), 420–436.

Box, G. E., W. G. Hunter, and J. S. Hunter (1978). Statistics for experimenters: an introduction to design, data analysis, and model building.

Burgers, G., P. Jan van Leeuwen, and G. Evensen (1998). Analysis scheme in the ensemble kalman filter. *Monthly weather review 126*(6), 1719–1724.

Burroughs, E., E. Coutsias, and L. Romero (2005). A reduced-order partial differential equation model for the flow in a thermosyphon. *Journal of Fluid Mechanics 543*, 203–238.

Carlton, J. (2009, dec). Keeping it frozen. *The Wall Street Journal Environment*.

Cherry, B., Z. Dan, and A. Lustgarten (2006). Next stop, lhasa. *Fortune 153*, 154.

Clayton, A., A. Lorenc, and D. Barker (2012). Operational implementation of a hybrid ensemble/4d-var global data assimilation system at the met office. *Quarterly Journal of the Royal Meteorological Society*.

Creveling, H., D. PAZ, J. Baladi, and R. Schoenhals (1975). Stability characteristics of a single-phase free convection loop. *Journal of Fluid Mechanics 67*(part 1), 65–84.

Danforth, C. M. (2009, March). 2013. `http://mpe2013.org/2013/03/17/chaos-in-an-atmosphere-hanging-on-a-wall/`.

Danforth, C. M., E. Kalnay, and T. Miyoshi (2007). Estimating and correcting global weather model error. *Monthly weather review 135*(2), 281–299.

Dee, D. P. and A. M. Da Silva (1998). Data assimilation in the presence of forecast bias. *Quarterly Journal of the Royal Meteorological Society 124*(545), 269–295.

Desrayaud, G., A. Fichera, and M. Marcoux (2006). Numerical investigation of natural circulation in a 2d-annular closed-loop thermosyphon. *International journal of heat and fluid flow 27*(1), 154–166.

Detman, R. F. (1968, July 16). Thermos iphon depp pool reactor. US Patent 3,393,127.

Edwards, A. W. F. (1984). Likelihood: An account of the statistical concept of likelihood and its application to scientific inferences.

Ehrhard, P. and U. Müller (1990). Dynamical behaviour of natural convection in a single-phase loop. *Journal of Fluid mechanics 217*, 487–518.

Evensen, G. (2003). The ensemble kalman filter: Theoretical formulation and practical implementation. *Ocean dynamics 53*(4), 343–367.

Ferziger, J. H. and M. Perić (1996). *Computational methods for fluid dynamics*, Volume 3. Springer Berlin.

Ginsberg, J., M. H. Mohebbi, R. S. Patel, L. Brammer, M. S. Smolinski, and L. Brilliant (2008). Detecting influenza epidemics using search engine query data. *Nature 457*(7232), 1012–1014.

Glenn, D. (2013, may). Characterizing weather in a thermosyphon: an atmosphere that hangs on a wall. Undergraduate Honors Thesis, University of Vermont.

Gorman, M. and P. Widmann (1984). Chaotic flow regimes in a convection loop. *Phys. Rev. Lett.* (52), 2241–2244.

Gorman, M., P. Widmann, and K. Robbins (1986). Nonlinear dynamics of a convection loop: a quantitative comparison of experiment with theory. *Physica D* (19), 255–267.

Hamill, T. M., J. S. Whitaker, and C. Snyder (2001). Distance-dependent filtering of background error covariance estimates in an ensemble kalman filter. *Monthly Weather Review 129*(11), 2776–2790.

Harris, K. D., E. H. Ridouane, D. L. Hitt, and C. M. Danforth (2011). Predicting flow reversals in chaotic natural convection using data assimilation. *arXiv preprint arXiv:1108.5685*.

Hsiang, S. M., M. Burke, and E. Miguel (2013). Quantifying the influence of climate on human conflict. *Science 341*(6151).

Hunt, B. R., E. J. Kostelich, and I. Szunyogh (2007). Efficient data assimilation for spatiotemporal chaos: A local ensemble transform kalman filter. *Physica D: Nonlinear Phenomena 230*(1), 112–126.

Issa, R. I. (1986). Solution of the implicitly discretised fluid flow equations by operator-splitting. *Journal of Computational physics 62*(1), 40–65.

Jasak, H., A. Jemcov, and Z. Tukovic (2007). Openfoam: A c++ library for complex physics simulations. In *International Workshop on Coupled Methods in Numerical Dynamics, IUC, Dubrovnik, Croatia*, pp. 1–20.

Jiang, Y. and M. Shoji (2003). Spatial and temporal stabilities of flow in a natural circulation loop: influences of thermal boundary condition. *J Heat Trans.* (125), 612–623.

Johnson, T. (2009, sep). 'toy climate' simulates hurricanes. *The Burlington Free Press Vermont*, C.1.

Kalman, R. and R. Bucy (1961). New results in linear prediction and filtering theory. *Trans. AMSE J. Basic Eng. D 83*, 95–108.

Kalnay, E. (2003). *Atmospheric modeling, data assimilation, and predictability*. Cambridge university press.

Kalnay, E., H. Li, T. Miyoshi, S.-C. YANG, and J. BALLABRERA-POY (2007). 4-d-var or ensemble kalman filter? *Tellus A 59*(5), 758–773.

Keller, J. B. (1966). Periodic oscillations in a model of thermal convection. *J. Fluid Mech 26*(3), 599–606.

Kwant, W. and C. Boardman (1992). Prismliquid metal cooled reactor plant design and performance. *Nuclear engineering and design 136*(1), 111–120.

Li, H., E. Kalnay, T. Miyoshi, and C. M. Danforth (2009). Accounting for model errors in ensemble data assimilation. *Monthly Weather Review 137*(10), 3407–3419.

Lorenc, A. (1981). A global three-dimensional multivariate statistical interpolation scheme. *Monthly Weather Review 109*(4), 701–721.

Lorenc, A. C. (1986). Analysis methods for numerical weather prediction. *Quarterly Journal of the Royal Meteorological Society 112*(474), 1177–1194.

Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the atmospheric sciences 20*(2), 130–141.

Louisos, W. F., D. L. Hitt, and C. M. Danforth (2013). Chaotic flow in a 2d natural convection loop with heat flux boundaries. *International Journal of Heat and Mass Transfer 61*, 565–576.

Mitchell, L. (2012). *Incorporating climatological information into ensemble data assimilation*. Ph. D. thesis, The University of Sydney.

Navier, C. (1823). Mémoire sur les lois du mouvement des fluides. *Mémoires de lAcadémie Royale des Sciences de lInstitut de France 6*, 389–440.

Navon, I. M., X. Zou, J. Derber, and J. Sela (1992). Variational data assimilation with an adiabatic version of the nmc spectral model. *Monthly weather review 120*(7), 1433–1446.

Ott, E., B. R. Hunt, I. Szunyogh, A. V. Zimin, E. J. Kostelich, M. Corazza, E. Kalnay, D. Patil, and J. A. Yorke (2004). A local ensemble kalman filter for atmospheric data assimilation. *Tellus A 56*(5), 415–428.

Patankar, S. V. and D. B. Spalding (1972). A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer 15*(10), 1787–1806.

Purser, R. J. (1984). A new approach to the optimal assimilation of meteorological data by iterative bayesian analysis. In *Conference on Weather Forecasting and Analysis, 10 th, Clearwater Beach, FL*, pp. 102–105.

Richardson, L. F. and S. Chapman (1922). *Weather prediction by numerical process*. Dover publications New York, 1965 Reprint.

Ridouane, E. H., C. M. Danforth, and D. L. Hitt (2010). A 2-d numerical study of chaotic flow in a natural convection loop. *International Journal of Heat and Mass Transfer 53*(1), 76–84.

Saltzman, B. (1962). Finite amplitude free convection as an initial value problem-i. *Journal of the Atmospheric Sciences 19*(4), 329–341.

Savely, R., B. Cockrell, , and S. Pines (1972). Apollo experience report - onboard navigational and alignment software. *Technical Report*.

Siouris, G. M., G. Chen, and J. Wang (1997). Tracking an incoming ballistic missile using an extended interval kalman filter. *Aerospace and Electronic Systems, IEEE Transactions on 33*(1), 232–240.

Sornette, D. and W.-X. Zhou (2006). Predictability of large future changes in major financial indices. *International Journal of Forecasting 22*(1), 153–168.

Stokes, S. G. G. (1846). *On the Friction of Fluids in Motion, and the Equilibrium and Motion of Elastic Solids*.

Talagrand, O. (1997). Assimilation of observations, an introduction. *JOURNAL-METEOROLOGICAL SOCIETY OF JAAN SERIES 2 75*, 81–99.

Thompson, P. (1990). Charney and the revival of nwp. *The Atmosphere - a challenge*, 93–119.

Welander, P. (1995). On the oscillatory instability of a differentially heated fluid loop. *International Geophysics Series 59*.

Xu, J. and D. J. Goering (2008). Experimental validation of passive permafrost cooling systems. *Cold Regions Science and Technology 53*(3), 283 – 297. ¡ce:title¿The QinghaiTibet Railroad: A milestone project and its environmental impact¡/ce:title¿.

Yang, S.-C., D. Baker, H. Li, K. Cordes, M. Huff, G. Nagpal, E. Okereke, J. Villafane, E. Kalnay, and G. S. Duane (2006). Data assimilation as synchronization of truth and model: Experiments with the three-variable lorenz system*. *Journal of the atmospheric sciences 63*(9), 2340–2354.

Yuen, P. and H. Bau (1999). Optimal and adaptive control of chaotic convection. *Phys. Fluids* (11), 1435–1448.

Appendix A: Parameters

Table A.1: Summary of Lorenz '63 parameters

Parameter	Value	Interpretation, if any
σ	10	
b	8/3	
r	28	

Table A.2: EKF Covariance Inflation

Window Length	**Additive Inflataion**	**Multiplicative Inflation**
30s	0.05	0.15

Appendix B: Derivation of CFD Solving Code

Closely following the original derivation of Stokes (Stokes 1846), I present a full derivation of the main equations governing the flow of water at temperatures near 300K (an incompressible, heat-conducting, Newtonian fluid), and detail their implementation in a finite volume numerical solver. Detailed equations follow from my notes based on the reference work of *Computational Fluid Dynamics* written by Anderson et al for the von Karman Institute lectures (Anderson et al. 1995). From the available approaches, I consider a fixed finite volume to derive the equations used in OpenFOAM.

B.1 Continuity Equation

Considering a finite volume element with side lengths $\Delta x, \Delta y$ and Δz. The amount of mass that enters any given face on this volume is a function of the fluid density ρ, the velocity tangent to this face, and the area of the face. For the a side with edges specified by both Δx and Δy, let the velocity tangent to this face be u and this mass flow in this side is equal to

$$\rho u \Delta x \Delta y.$$

Assuming u is positive in this direction, the mass flux out of the opposite side needs to reflect possible changes in velocity u and density ρ through the volume and can be written

$$-(\rho + \Delta\rho)(u + \Delta u)\Delta x \Delta y.$$

Similarly for the other two directions tangent to our volume, we assign velocities v and w with incoming mass

$$\rho v \Delta x \Delta z$$
$$\rho w \Delta y \Delta z$$

and outgoing mass

$$\begin{aligned} &-(\rho + \Delta\rho)(v + \Delta v)\Delta x \Delta z \\ &-(\rho + \Delta\rho)(w + \Delta w)\Delta y \Delta z. \end{aligned}$$

The rate of mass accumulation in the volume,

$$\Delta\rho(\Delta x \Delta y \Delta z)/\Delta t,$$

must be equal to the sum of the mass entering and leaving the volume (given by the six equations above). Equating these, with a little cancellation and dividing by $(\Delta x \Delta y \Delta z)$ we are left with

$$\frac{\Delta\rho}{\Delta t} + \frac{\Delta(\rho u)}{\Delta x} + \frac{\Delta(\rho v)}{\Delta y} + \frac{\Delta(\rho w)}{\Delta z} = 0. \tag{B.1}$$

As $\Delta t \to 0$, this can be written in terms of the partial derivatives

$$\frac{\partial \rho}{\partial t} + \frac{\partial(\rho u)}{\partial x} + \frac{\partial(\rho v)}{\partial y} + \frac{\partial(\rho w)}{\partial z} = 0. \tag{B.2}$$

For an incompressible fluid, we have that $\partial\rho/\partial\{t, x, y, z\} = 0$ such the continuity equation becomes

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = 0. \tag{B.3}$$

B.2 Momentum Equation

We now consider the second conservation law: the conservation of momentum. This states that the rate of change of momentum must equal the net momentum flux into the control volume in addition to any external forces (e.g. gravity) on the control volume. Again consider a small but finite volume element with side lengths $\Delta x, \Delta y$ and Δz. Since the conservation of momentum applies in x, y and z directions, without loss of generality I consider only the x direction.

The rate of change of momentum with respect to time in our volume element is

$$\frac{\partial}{\partial t}(\rho u)\,\Delta x \Delta y \Delta z.$$

The momentum flux into the volume in the x direction is the mass flux times x-velocity, since momentum is mass$\times$velocity. Recall the mass flux is given by $\rho u \times A$ for A the area orthogonal to x, in this case $\Delta y \Delta z$. Therefore the momentum flux is

$$u \times \rho u(\Delta y \Delta z).$$

Similarly, with the additional consideration of the change within the volume, the momentum flux leaving through the opposite side is

$$-\left(u\rho u + \frac{\partial}{\partial x}\left(u\rho u\right)\Delta x\right)\Delta y\Delta z.$$

The y and z direction momemtum flux (of x direction momentum) is found in the same way and we have the fluxes in those directions as

$$\begin{aligned} u \times \rho v(\Delta x\Delta z) \quad &\& \quad -\left(u\rho v + \frac{\partial}{\partial y}\left(u\rho v\right)\Delta y\right)\Delta x\Delta z \\ u \times \rho w(\Delta x\Delta y) \quad &\& \quad -\left(u\rho w + \frac{\partial}{\partial z}\left(u\rho w\right)\Delta z\right)\Delta x\Delta y. \end{aligned}$$

Equating the change in internal momentum with the sum of the fluxes, and the sum of the external forces in the x direction $\mathbf{F}_1$, we have the most basic form of the momentum equation:

$$\begin{aligned} \frac{\partial}{\partial t}\left(\rho u\right)\Delta x\Delta y\Delta z = \frac{\partial}{\partial t}\left(\rho u\right)\Delta x\Delta y\Delta z &- \left(u\rho u + \frac{\partial}{\partial x}\left(u\rho u\right)\Delta x\right)\Delta y\Delta z + u\rho v(\Delta x\Delta z) \\ &- \left(u\rho v + \frac{\partial}{\partial y}\left(u\rho v\right)\Delta y\right)\Delta x\Delta z + u\rho w(\Delta x\Delta y) \\ &- \left(u\rho w + \frac{\partial}{\partial z}\left(u\rho w\right)\Delta z\right)\Delta x\Delta y + \sum \mathbf{F}_1. \end{aligned}$$

Canceling the first terms from each flux we are left

$$\begin{aligned} \frac{\partial}{\partial t}\left(\rho u\right)\Delta x\Delta y\Delta z = &-\frac{\partial}{\partial x}\left(u\rho u\right)\Delta x\Delta y\Delta z - \frac{\partial}{\partial y}\left(u\rho v\right)\Delta y\Delta x\Delta z \\ &- \frac{\partial}{\partial z}\left(u\rho w\right)\Delta z\Delta x\Delta y + \sum \mathbf{F}_1. \end{aligned}$$

Pulling out the $\Delta x \Delta y \Delta z$ and moving just the forces to the right we have

$$\Delta x \Delta y \Delta z \left(\frac{\partial}{\partial t} (\rho u) + \frac{\partial}{\partial x} (u \rho u) + \frac{\partial}{\partial y} (u \rho v) + \frac{\partial}{\partial z} (u \rho w) \right) = \sum \mathbf{F}_1.$$

Applying the product rule to the partial derivatives with respect to x, y and z we find the continuity equation which we know to be zero, and are left

$$\Delta x \Delta y \Delta z \left(\rho \frac{\partial u}{\partial t} + \rho u \frac{\partial u}{\partial x} + \rho v \frac{\partial u}{\partial y} + \rho w \frac{\partial u}{\partial z} \right) = \sum \mathbf{F}_1.$$

The sum of the forces in the x direction includes the force of gravity that acts on the mass of the entire volume ($g_1 \times \rho \Delta x \Delta y \Delta z$) and the surface stress. We assume that gravity acts only in the z direction where the term $g_3 \times \rho \Delta x \Delta y \Delta z$ shows up in the sum of the forces and is otherwise zero. The x-direction force of the stresses acting on the volume is the product of the stress and the area on which it acts, and in Cartesian coordinates this is either direct or shear stress. The x-normal stress, which we denote s_{xx} has forces on both sides tangent to x given by

$$s_{xx} \Delta y \Delta z \quad \& \quad \left(s_{xx} + \frac{\partial s_{xx}}{\partial x} \Delta x \right) \Delta y \Delta z$$

for which the sum is

$$\frac{\partial s_{xx}}{\partial x} \Delta x \Delta y \Delta z.$$

The forces of the shear stress on the faces planar to x are similarly

$$\frac{\partial s_{yx}}{\partial x} \Delta x \Delta y \Delta z \quad \& \quad \frac{\partial s_{zx}}{\partial x} \Delta x \Delta y \Delta z.$$

Since the pressure p does not apply shear stress, p is only a part of s_{xx} and not s_{yx}, s_{zx}. In each direction we denote the viscous stress as τ_{ij} for i and j the directions x, y, and z. Thus for general fluids the sum of the forces in the x direction is

$$\left(\frac{\partial p}{\partial x} + \frac{\partial \tau_{xx}}{\partial x} + \frac{\partial \tau_{yx}}{\partial y} + \frac{\partial \tau_{zx}}{\partial z}\right) \Delta x \Delta y \Delta z.$$

Since the thermosyphon is filled with water, not blood, we can happily assume that the fluid is Newtonian, and the rapture has been avoided. This means that we can relate the viscous stress to the local rate of deformation linearly, by the viscocity μ:

$$\begin{aligned}
\tau_{xx} &= 2\mu \frac{\partial u}{\partial x} \\
\tau_{yx} &= \mu \left(\frac{\partial v}{\partial x} + \frac{\partial u}{\partial y}\right) \\
\tau_{zx} &= \mu \left(\frac{\partial w}{\partial x} + \frac{\partial u}{\partial z}\right)
\end{aligned}$$

Putting this together for our Newtonian fluid we have by the equality of mixed partials and the continuity equation we have the sum of all the forces given by

$$\begin{aligned}
&\left(-\frac{\partial p}{\partial x} + 2\mu \frac{\partial u}{\partial x^2} + \mu \frac{\partial}{\partial y}\frac{\partial v}{\partial x} + \mu \frac{\partial}{\partial y}\frac{\partial u}{\partial y} + \mu \frac{\partial}{\partial z}\frac{\partial u}{\partial z} + \mu \frac{\partial}{\partial z}\frac{\partial w}{\partial x}\right) \Delta x \Delta y \Delta z \\
&= \left(-\frac{\partial p}{\partial x} + \mu \frac{\partial u}{\partial x^2} + \mu \frac{\partial u}{\partial y^2} + \mu \frac{\partial u}{\partial z^2} + \mu \frac{\partial u}{\partial x^2} + \mu \frac{\partial}{\partial x}\frac{\partial v}{\partial y} + \mu \frac{\partial}{\partial x}\frac{\partial w}{\partial z}\right) \Delta x \Delta y \Delta z \\
&= \left(-\frac{\partial p}{\partial x} + \mu \frac{\partial u}{\partial x^2} + \mu \frac{\partial u}{\partial y^2} + \mu \frac{\partial u}{\partial z^2} + \mu \frac{\partial}{\partial x}\left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z}\right)\right) \Delta x \Delta y \Delta z \\
&= \left(-\frac{\partial p}{\partial x} + \mu \frac{\partial u}{\partial x^2} + \mu \frac{\partial u}{\partial y^2} + \mu \frac{\partial u}{\partial z^2}\right) \Delta x \Delta y \Delta z
\end{aligned}$$

Thus the momemtum equation for x is

$$\rho\left(\frac{\partial u}{\partial t}+u\frac{\partial u}{\partial x}+v\frac{\partial u}{\partial y}+w\frac{\partial u}{\partial z}\right)=-\frac{\partial p}{\partial x}+\mu\frac{\partial u}{\partial x^2}+\mu\frac{\partial u}{\partial y^2}+\mu\frac{\partial u}{\partial z^2} \tag{B.4}$$

In tensor notation this is often written for all three equations (recall, the above is only for x):

$$\rho\frac{\partial u_i}{\partial t}+\frac{\partial}{\partial x_j}\left(u_j u_i\right)=-\frac{\partial p}{\partial x_i}+\mu\frac{\partial u_i}{\partial x_j^2}+\rho g_i. \tag{B.5}$$

Working our way towards OpenFOAM's implementation of this equation, write the filtered equation for averaged quantities of pressure $\bar{p}$, density $\bar{\rho}$ and velocity $\bar{u}$, and assume that the density is constant except for when multiplied by gravitational effects, writing $\bar{\rho}=\overline{\rho}/\rho_0$:

$$\rho_0\left(\frac{\partial \bar{u}_i}{\partial t}+\frac{\partial}{\partial x_j}\left(\bar{u}_j\bar{u}_i\right)\right)=-\frac{\partial \bar{p}}{\partial x_i}+\mu\frac{\partial \bar{u}_i}{\partial x_j^2}+\bar{\rho} g_i. \tag{B.6}$$

Since we will be relying on the use of a turbulence model, we reintroduce the stress tensor τ_{ij} and split τ_{ij} into the mean stress tensor τ_{ij} and the turbulent stress tensor τ^*_{ij}. In tensor notation the mean stress tensor can be written

$$\tau_{ij}=\mu\left(\left(\frac{\partial \bar{u}_i}{\partial x_j}+\frac{\partial \bar{u}_j}{\partial x_i}\right)-\frac{2}{3}\frac{\partial \bar{u}_k}{\partial x_k}\delta_{ij}\right)$$

for $\delta_{ij}=0$ if and only if $i=j$. Assume constant viscocity $\mu=\mu_0$, divide by ρ_0 and incorporate the latter for the stress tensor and we now have the momentum equation as

$$\frac{\partial \bar{u}_i}{\partial t}+\frac{\partial}{\partial x_j}\left(\bar{u}_j\bar{u}_i\right)=-\frac{\partial}{\partial x_i}\frac{\bar{p}}{\rho_0}+\frac{\partial}{\partial x_j}\left(\nu_0\left(\left(\frac{\partial \bar{u}_i}{\partial x_j}+\frac{\partial \bar{u}_j}{\partial x_i}\right)-\frac{2}{3}\left(\frac{\partial \bar{u}_k}{\partial x_k}\right)\delta_{ij}\right)-\tau^*_{ij}\right)+\bar{\rho} g_i$$

where $\nu_0 = \mu_0/\rho_0$. We can then decompose the turbulent stress tensor τ^*_{ij} further using R_{ij} to denote the Reynolds stress tensor

$$\tau^*_{ij} = R_{ij} = R^D_{ij} + \frac{2}{3}k\delta_{ij}$$

for R^D_{ij} the deviatoric part and $k = R_{ii}/2$ the turbulent kinetic energy. For LES this becomes the sub-grid kinetic energy. Note that OpenFOAM outputs the resolved kinematic pressure

$$\tilde{p} = \frac{\bar{p}}{\rho_0} + \frac{2}{3}k$$

and uses the Boussinesq approximation for the last term

$$g_i\frac{\bar{\rho}}{\rho_0} = g_i\left(1 + \frac{\bar{\rho} - \rho_0}{\rho_0}\right) = g_i\rho_k = g_i\left(1 - \beta\left(\overline{T} - T_0\right)\right).$$

B.3 Energy Equation

Again following the derivation of Anderson, we derive the energy equation into the form used in OpenFOAM (Anderson Jr 2009). Since our problem deals with an incompressible fluid, we are concerned mainly with the temperature.

The first law of thermodynamics applied to a finite volume says that the rate of change of energy inside the fluid element is equal to the net flux of heat into the element plus the rate of work done by external forces on the volume. Writing the absolute internal thermal energy as e and the kinematic energy per unit mass as $v^2/2$ we have the rate of change of total internal energy as

$$\frac{\partial}{\partial t}\left(\rho\left(e + \frac{v^2}{2}\right)\right) \tag{B.7}$$

and the net transfer of energy through the control volume is

$$\frac{\partial}{\partial x}\left(u\rho\left(e+\frac{v^2}{2}\right)\right)+\frac{\partial}{\partial y}\left(v\rho\left(e+\frac{v^2}{2}\right)\right)+\frac{\partial}{\partial z}\left(w\rho\left(e+\frac{v^2}{2}\right)\right). \tag{B.8}$$

Heat flux $\dot{q}$ is defined to be positive for flux leaving the control volume, so we write the net heat flux entering the volume as

$$-\frac{\partial q_x}{\partial x}-\frac{\partial q_y}{\partial y}-\frac{\partial q_z}{\partial z}. \tag{B.9}$$

Since heat flux by thermal conduction is determined by Fourier's Law of Heat Conduction as

$$q_{\{x,y,z\}}=-k\frac{\partial T}{\partial\{x,y,z\}} \tag{B.10}$$

we write Equation B.9 as

$$k\left(\frac{\partial T}{\partial x^2}-\frac{\partial T}{\partial y^2}-\frac{\partial T}{\partial z^2}\right). \tag{B.11}$$

As before, the shear stress in the i direction on face j is denoted s_{ij} such that the net rate of work done by these stresses is the sum of their components

$$-\frac{\partial}{\partial x}\left(us_{xx}+vs_{xy}+ws_{xz}\right)-\frac{\partial}{\partial y}\left(us_{yx}+vs_{yy}+ws_{yz}\right)-\frac{\partial}{\partial z}\left(us_{zx}+vs_{zy}+ws_{zz}\right). \tag{B.12}$$

Putting this all together in tensor notation, the temperature (energy) equation is thus

$$\frac{\partial}{\partial t}\left(\rho e\right)+\frac{\partial}{\partial x_j}\left(\rho e u_j\right)=-k\frac{\partial T}{\partial x_k^2}. \tag{B.13}$$

Averaging this equation and separating the heat flux into the average and turbulent parts $q = \overline{q} + q^*$ we have

$$\frac{\partial}{\partial t}\left(\rho\overline{e}\right) + \frac{\partial}{\partial x_j}\left(\rho\overline{e}\overline{u}_j\right) = -\frac{\partial q_k^*}{\partial x_k} - \frac{\partial \overline{q}_k}{\partial x_k} \tag{B.14}$$

B.4 Implementation

The PISO (Pressure-Implicit with Splitting of Operators) algorithm derives from the work of (Issa 1986), and is complementary to the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) (Patankar and Spalding 1972) iterative method. The main difference of the PISO and SIMPLE algorithms is that in the PISO, no under-relaxation is applied and the momentum corrector step is performed more than once (Ferziger and Perić 1996). They sum up the algorithm in nine steps:

- Set the boundary conditions
- Solve the discretized momentum equation to compute an intermediate velocity field
- Compute the mass fluxes at the cell faces
- Solve the pressure equation
- Correct the mass fluxes at the cell faces
- Correct the velocity with respect to the new pressure field
- Update the boundary conditions
- Repeat from step #3 for the prescribed number of times
- Repeat (with increased time step).

The solver itself has 647 dependencies, of which I present only a fraction. The main code is straight forward, relying on include statements to load the libraries and equations to be solved.

```
#include "fvCFD.H"
#include "singlePhaseTransportModel.H"
#include "RASModel.H"
#include "radiationModel.H"
#include "fvIOoptionList.H"
#include "pimpleControl.H"
```

The main function is then

```
int main(int argc, char *argv[])
{
    #include "setRootCase.H"
    #include "createTime.H"
    #include "createMesh.H"
    #include "readGravitationalAcceleration.H"
    #include "createFields.H"
    #include "createIncompressibleRadiationModel.H"
    #include "createFvOptions.H"
    #include "initContinuityErrs.H"
    #include "readTimeControls.H"
    #include "CourantNo.H"
    #include "setInitialDeltaT.H"

    pimpleControl pimple(mesh);
```

We then enter the main loop. This is computed for each time step, prescribed before the solver is applied. Note that the capacity is available for adaptive time steps, choosing to keep the Courant number below some threshold, but I do not use this. For the distributed ensemble of model runs, it is important that each model complete in nearly the same time, so that the analysis is not waiting on one model and therefore under-utilizing the available resources.

```
while (runTime.loop())
{
    Info<< "Time = " << runTime.timeName() << nl << endl;

    #include "readTimeControls.H"
    #include "CourantNo.H"
    #include "setDeltaT.H"

    // --- Pressure-velocity PIMPLE corrector loop
    while (pimple.loop())
    {
        #include "UEqn.H"
        #include "TEqn.H"

        // --- Pressure corrector loop
        while (pimple.correct())
        {
            #include "pEqn.H"
        }

        if (pimple.turbCorr())
        {
            turbulence->correct();
        }
    }

    runTime.write();
```

Opening up the equation for U we see that Equation

```
// Solve the momentum equation
fvVectorMatrix UEqn
(
    fvm::ddt(U)
  + fvm::div(phi, U)
  + turbulence->divDevReff(U)
 ==
    fvOptions(U)
```

```
);
UEqn.relax();
fvOptions.constrain(UEqn);
if (pimple.momentumPredictor())
{
    solve
    (
        UEqn
     ==
        fvc::reconstruct
        (
            (
              - ghf*fvc::snGrad(rhok)
              - fvc::snGrad(p_rgh)
            )*mesh.magSf()
        )
    );
    fvOptions.correct(U);
}
```

Solving for T is

```
{
    alphat = turbulence->nut()/Prt;
    alphat.correctBoundaryConditions();
    volScalarField alphaEff("alphaEff", turbulence->nu()/Pr + alphat);
    fvScalarMatrix TEqn
    (
        fvm::ddt(T)
      + fvm::div(phi, T)
      - fvm::laplacian(alphaEff, T)
     ==
        radiation->ST(rhoCpRef, T)
      + fvOptions(T)
    );
    TEqn.relax();
    fvOptions.constrain(TEqn);
    TEqn.solve();
```

```
    radiation->correct();
    fvOptions.correct(T);
    rhok = 1.0 - beta*(T - TRef); // Boussinesq approximation
}
```

Finally, we solve for the pressure p in “pEqn.H”:

```
{
    volScalarField rAU("rAU", 1.0/UEqn.A());
    surfaceScalarField rAUf("Dp", fvc::interpolate(rAU));
    volVectorField HbyA("HbyA", U);
    HbyA = rAU*UEqn.H();
    surfaceScalarField phig(-rAUf*ghf*fvc::snGrad(rhok)*mesh.magSf());
    surfaceScalarField phiHbyA
    (
        "phiHbyA",
        (fvc::interpolate(HbyA) & mesh.Sf())
      + fvc::ddtPhiCorr(rAU, U, phi)
      + phig
    );
    while (pimple.correctNonOrthogonal())
    {
        fvScalarMatrix p_rghEqn
        (
            fvm::laplacian(rAUf, p_rgh) == fvc::div(phiHbyA)
        );
        p_rghEqn.setReference(pRefCell, getRefCellValue(p_rgh, pRefCell));
        p_rghEqn.solve(mesh.solver(p_rgh.select(pimple.finalInnerIter())));
        if (pimple.finalNonOrthogonalIter())
        {
            // Calculate the conservative fluxes
            phi = phiHbyA - p_rghEqn.flux();
            // Explicitly relax pressure for momentum corrector
            p_rgh.relax();
            // Correct the momentum source with the pressure gradient flux
            // calculated from the relaxed pressure
            U = HbyA + rAU*fvc::reconstruct((phig - p_rghEqn.flux())/rAUf);
            U.correctBoundaryConditions();
```

```
        }
    }
    #include "continuityErrs.H"
    p = p_rgh + rhok*gh;
    if (p_rgh.needReference())
    {
        p += dimensionedScalar
        (
            "p",
            p.dimensions(),
            pRefValue - getRefCellValue(p, pRefCell)
        );
        p_rgh = p - rhok*gh;
    }
}
```

The final operation being the conversion of pressure to hydrostatic pressure,

$$p_{\text{rgh}} = p - \rho_k g_h.$$

This "pEqn.H" is then re-run until convergence is achieved, and the PISO loop begins again.

Appendix C: Code

☐ either find a better way to include a summarized version of code, or don't include at all

The core components of the code necessary to reproduce all results herein is as follows. None of OpenFOAM version 2.2.1 is included, as it is freely available for download online.

```
classdef OpenFOAM<handle
    %OpenFOAM: class for an OpenFOAM run
    %   Goal is to implement a class for which MATLAB can
    %   control and run an OpenFOAM instance
    %
    %   USAGE:
    %       -initialize an OpenFOAM directory, read to be run, at 'testCase'
    %         ens1 = OpenFOAM();
    %         ens1.init('testCase');
    %       -set the T variable for the initial run
    %         ens1.T = 300*ones(40832,1);
    %       -run OpenFOAM on this case
    %         ens1.run();
    %       -run more into the future
    %         ens1.ETIME = 40;
    %         ens1.run();
    %       -go back and read T from time 30 (note they need to be divisors of 5 now,
         set in caseDir/system/controlDict)
    %         ens1.read(30)

    properties
        %basics
        x
```

```
        dim
        tstep = 1;
        time = 0;
        window = 20;
        params % dummy for now


        foamLab = '/users/a/r/areagan/work/2013/data-assimilation/OpenFOAM/foamLab.sh'
            ;
        TIME=0;
        ETIME=20;
        DIR='testCase'
        FLUX=10000; % for flux BC
        BCbottom = '340'; % for fixed value BC
        BCtop = '290';
        BC = 'fixedValue';
        TURB = 'off';
        BASE = '2D-foamLabSmall';
        TURBMODEL = 'kEpsilon';
        T
        U
        p
        p_rgh
        Tstep = 1;
        WRITEp = 6;
        WRITEint = 20;
        VALUE = [];
    end % properties

    methods
        function self = OpenFOAM(varargin)
            % intialize the class
            if nargin > 0
                self.BASE = varargin{1};
            end
            switch self.BASE
                case '2D-foamLabSmall'
```

```
                self.dim = 600;
            case '2D-foamLabBase'
                self.dim = 36000;
            case '2D-foamLabBase-mesh40000'
                self.dim = 40000;
            otherwise
                warning('chosen case does not have a predifined dimension. edit 
                    class to include it');
        end
        self.T = 300*ones(self.dim,1);
        self.U = zeros(self.dim,3); %1e-6*ones(40832,3);
        self.p = zeros(self.dim,1); %1e-6*ones(40832,1);
        self.p_rgh = zeros(self.dim,1); %1e-6*ones(40832,1);
        self.vectorize('in');
    end % constructor
    function init(self,varargin)
        if nargin > 1
            self.DIR = varargin{1};
        end
        % intialize a a clean dir
        system(sprintf('%s -i -d %s -B %s',self.foamLab,self.DIR,self.BASE));
        % !self.foamLab -i -d self.DIR
        %self.setFlux
    end % init
    function read(self,time,varargin)
        % read in a specific time variable
        % first: have foamLab write it as a csv
        caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',self.
            DIR);
        system(sprintf('%s -r %g -d %s -D %d',self.foamLab,time,caseDir,self.dim))
            ;
        self.T = csvread(sprintf('%s/%g/T.csv',caseDir,time));
        self.U = csvread(sprintf('%s/%g/U.csv',caseDir,time));
        self.p = csvread(sprintf('%s/%g/p.csv',caseDir,time));
        self.p_rgh = csvread(sprintf('%s/%g/p_rgh.csv',caseDir,time));
        self.vectorize('in');
        %return T
```

```
        end % read
        function write(self,time,varargin)
            % write out variable
            self.vectorize('out');
            caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',self.
                DIR);
            csvwrite(sprintf('%s/%g/T.csv',caseDir,time),self.T);
            csvwrite(sprintf('%s/%g/U.csv',caseDir,time),self.U);
            csvwrite(sprintf('%s/%g/p.csv',caseDir,time),self.p);
            csvwrite(sprintf('%s/%g/p_rgh.csv',caseDir,time),self.p_rgh);
            system(sprintf('%s -W %g -d %s',self.foamLab,time,caseDir));
        end % write
        function slice(self,num,varargin)
            % 'num' is the slice ID
            % this can be 1-8, representing the obviousness below
            % the top slice Cell IDs from paraview
            right = [35098 35156 35213 35268 35321 35372 35421 35468 35513 35556 35597
                 35636 35673 35708 35741 35772 35801 35828 35853 35876 35897 35916
                35933 35948 35961 35972 35981 35988 35993 35996];
            oneThirty = 0:1:29;
            top = 17218:58:18900;
            tenThirty = 0:1:29;
            left = [ 0 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361
                 400 441 484 529 576 625 676 729 784 841];
            sevenThirty = 0:1:29;
            bottom = 17159:58:18841;
            fourThirty = 0:1:29;
            % add 1 bc matlab indexes at 1
            allCellID = [right+1; oneThirty+1; top+1; tenThirty+1; left+1; sevenThirty
                +1; bottom+1; fourThirty+1];
            %disp(allCellID)
            cellID=allCellID(num,:);
            %disp(cellID)
            self.VALUE = [mean(self.T(cellID)) mean(self.U(cellID,1)) mean(self.U(
                cellID,2)) mean(self.U(cellID,3)) mean(self.p(cellID)) mean(self.p_rgh
                (cellID))];
        end % slice
```

```
        function run(self,varargin)
            % convert relevant variables to foam-readable and insert them,
            % if necessary
            caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',self.
                DIR);

            % write out the current x before running
            self.write(self.time)

            % run the case...this is out of control
            system(sprintf('%s -x -d %s -t %g -h %s -g %s -e %g -l %g -w %g -c %g -b %
                s -q %s -Q %s -B %s -D %d',self.foamLab,caseDir,self.time,self.BCtop,
                self.BCbottom,self.time+self.window,self.tstep,self.WRITEp,(self.time+
                self.window)*self.tstep,self.BC,self.TURB,self.TURBMODEL,self.BASE,
                self.dim));
            % update time
            self.time = self.time+self.window;

            self.read(self.time);
        end % run
        function vectorize(self,direction)
            %% np is the number of variables used, order a permutation on their
                storage
            %% np = 6;
            %% order = 0:5;
            %% np = 3;
            %% order = [0 0 1 2 0 0];
            np = 1;
            order = 0:5;
            switch direction
                case 'in' % into the datavec
                    % disp('converting the variables self.T ... into self.x');
                    %% reset the self storage
                    self.x = ones(self.dim*np,1); % column vector
                    try
                        self.x((1:np:(self.dim*np+1-np))+order(1)) = self.T;
                    catch
```

```
                        saveint = randi(1000000);
                        fprintf('failed to set self.x, throwing the whole state space 
                            to %06d\n',saveint);
                        save(sprintf('setfailure%06d.mat',saveint),'self.dim','self.np
                            ','self.T');
                        warning('WARNING FAILED TO SET SELF.X');
                    end
                    %% self.x((1:np:(self.dim*np+1-np))+order(2)) = self.U(:,1);
                    %% self.x((1:np:(self.dim*np+1-np))+order(3)) = self.U(:,2);
                    %% self.x((1:np:(self.dim*np+1-np))+order(4)) = self.U(:,3);
                    %% self.x((1:np:(self.dim*np+1-np))+order(5)) = self.p;
                    %% self.x((1:np:(self.dim*np+1-np))+order(6)) = self.p_rgh;

                case 'out' % out of x into T,U,p,p_rgh
                    % this will never be a problem
                    % if length(self.x) > 1 % check that something is there

                    self.T = self.x((1:np:(self.dim*np+1-np))+order(1));
                    %% self.U(:,1) = self.x((1:np:self.dim*np)+order(2));
                    %% self.U(:,2) = self.x((1:np:(self.dim*np+1-np))+order(3));
                    %% self.U(:,3) = self.x((1:np:(self.dim*np+1-np))+order(4));
                    %% self.p = self.x((1:np:self.dim*np)+order(5));
                    %% self.p_rgh = self.x((1:np:self.dim*np)+order(6));

                    % else
                    %     disp('there is nothing in x...did you mean to put in')
                    % end
            end % switch
        end % vectorize
        function destruct(self)
            % destroy the folder, for sanity's sake
            system(sprintf('\\rm -rf /users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s'
                ,self.DIR));
        end
    end % methods

end % classdef
```